机器视觉技术

刘晓芳 孙明竹
liuxiaofang@nankai.edu.cn
sunmz@nankai.edu.cn

机器人与信息自动化研究所
Institute of Robotics & Automatic Information System

南開大學
Nankai University

第三部分 深度学习图像处理

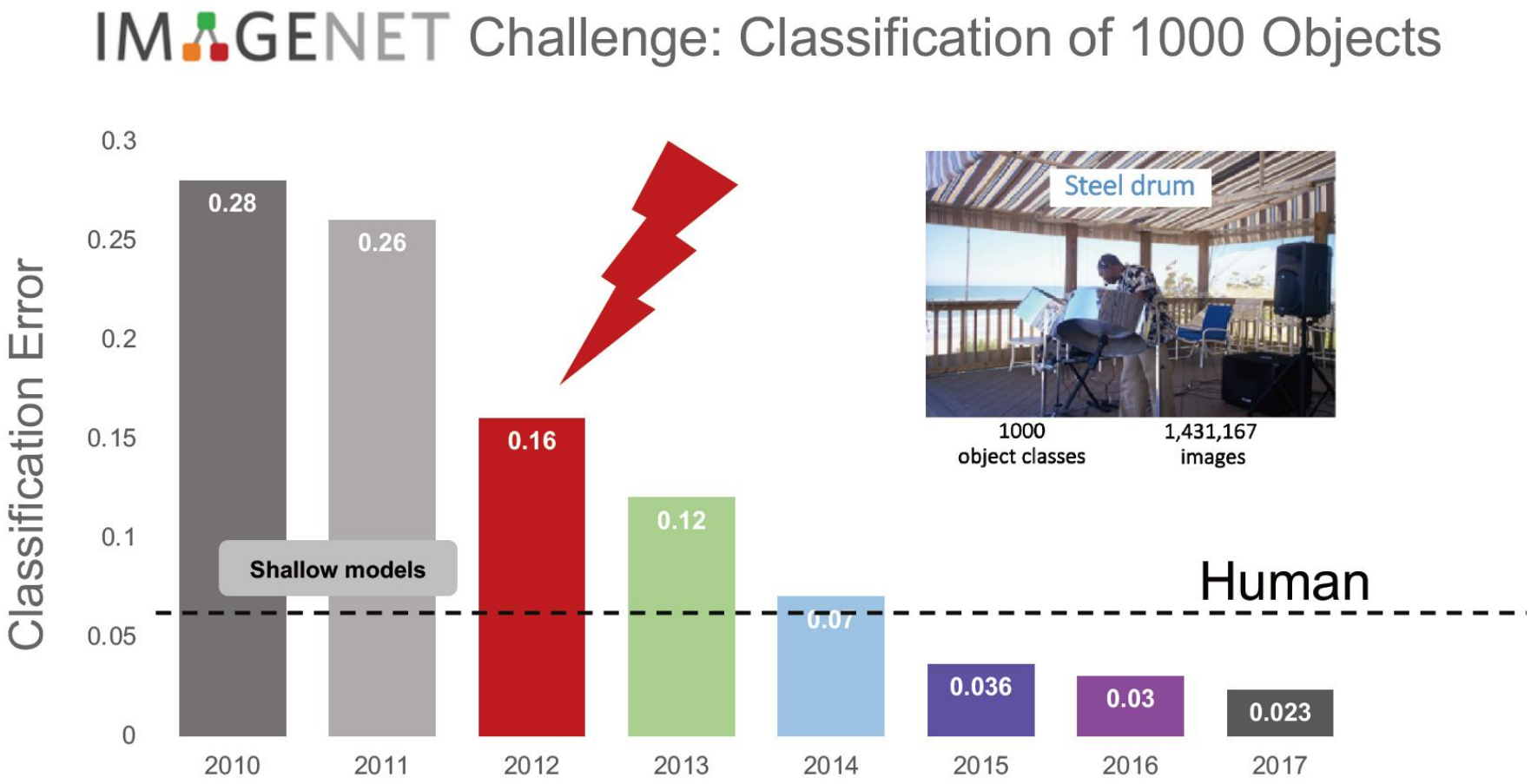
Deep Learning for Image Processing

主要内容

- **第六章 卷积神经网络**
- **第七章 图像分类**
  - 7.1 ImageNet挑战赛
  - 7.2 基于卷积神经网络的图像分类
    - AlexNet、VGGNet、GoogLeNet、ResNet
  - 7.3 分类网络发展
- **第八章 目标检测与图像分割**

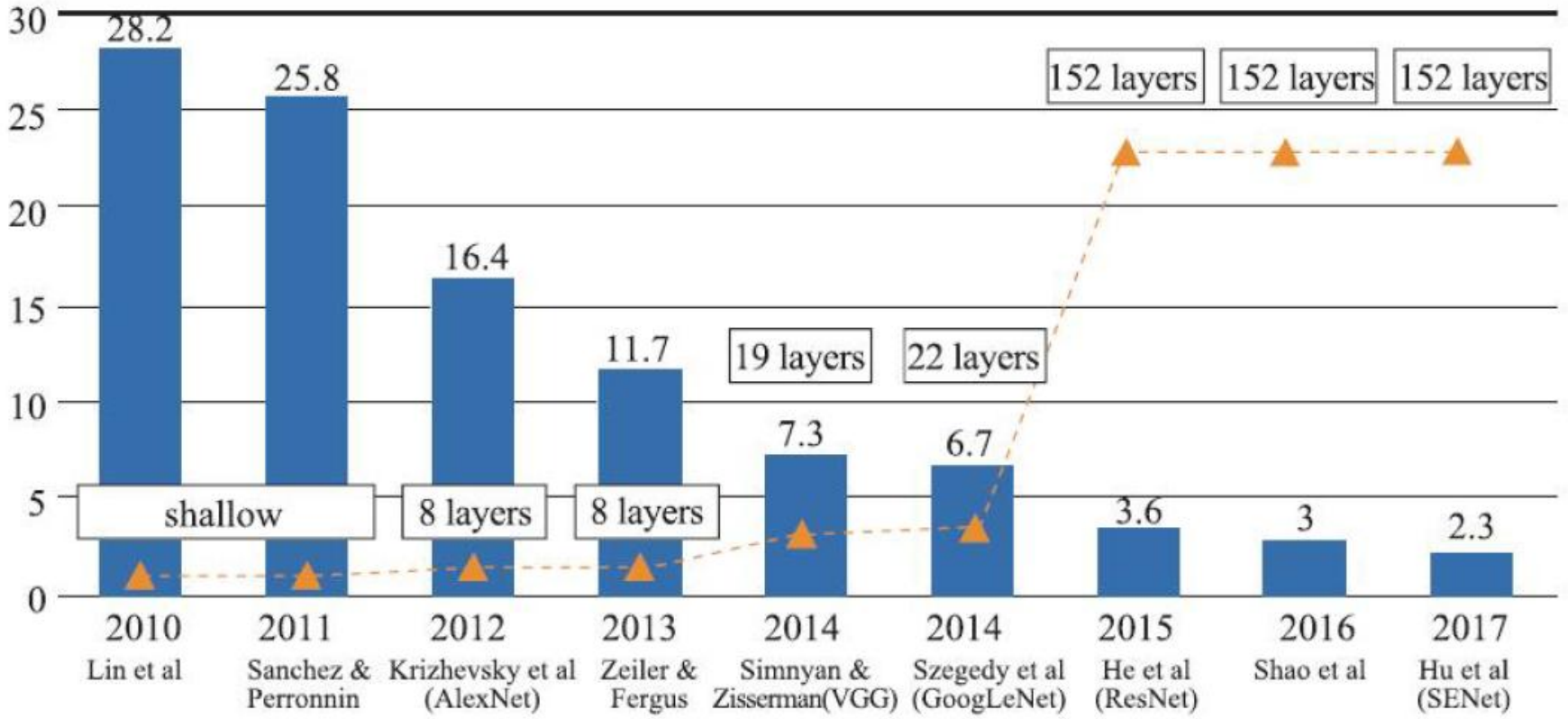
7.1 ImageNet挑战赛

训练集：1,281,167张已标注图像
验证集：50,000张已标注图像
测试集：100,000张未标注图像



Deng, J. et al. Fei-Fei, L. *CVPR*, 2009; Russakovsky et al. Fei-Fei, J. *IJCV*, 2012;

7.1 ImageNet挑战赛



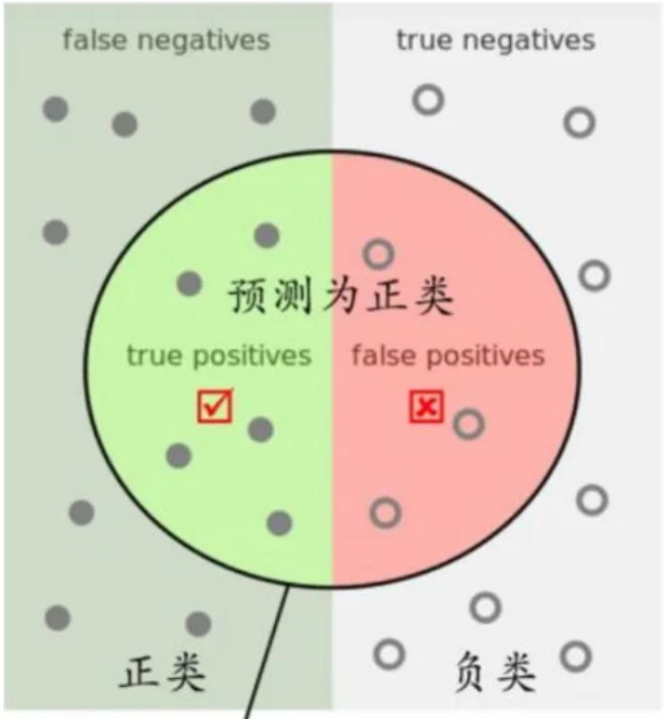

```
INPUT -> [[CONV -> RELU]*N -> POOL?]*M -> [FC -> RELU]*K -> FC
```

评价指标简介

混淆矩阵

TP(True Positive)=真阳性； FP(False Positive)=假阳性

FN(False Negative)=假阴性； TN(True Negative)=真阴性



- TP+FP+TN+FN：样本总数
- TP+FN：实际正样本数
- FP+TN：实际负样本数
- TP+FP：预测结果为正样本的总数，包括预测正确的和错误的
- TN+FN：预测结果为负样本的总数，包括预测正确的和错误的

评价指标简介

- **混淆矩阵**

  TP(True Positive)=真阳性； FP(False Positive)=假阳性

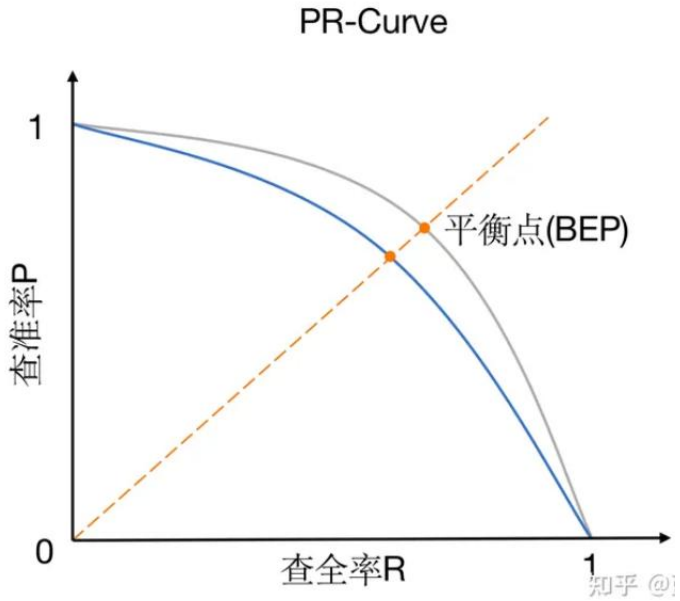
  FN(False Negative)=假阴性； TN(True Negative)=真阴性

- **评价指标**
  - 精确率 Precision = TP/(TP + FP)
  - 召回率 Recall = TP/(TP + FN)
  - 准确率 $Acc = \frac{TP+TN}{TP+FP+TN+FN}$

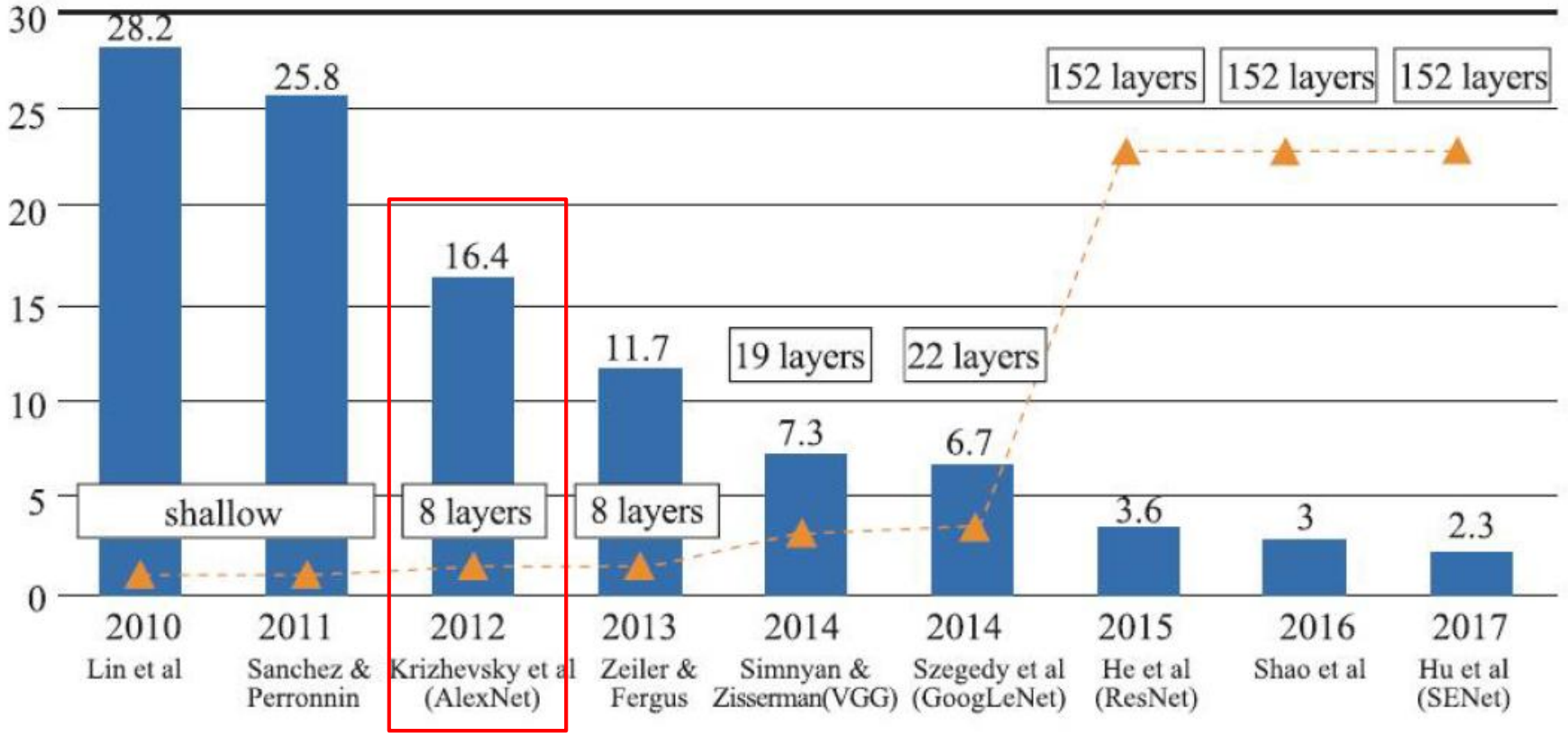
- TP+FN：实际正样本数
- TP+FP：预测结果为正样本的总数，包括预测正确的和错误的

评价指标简介

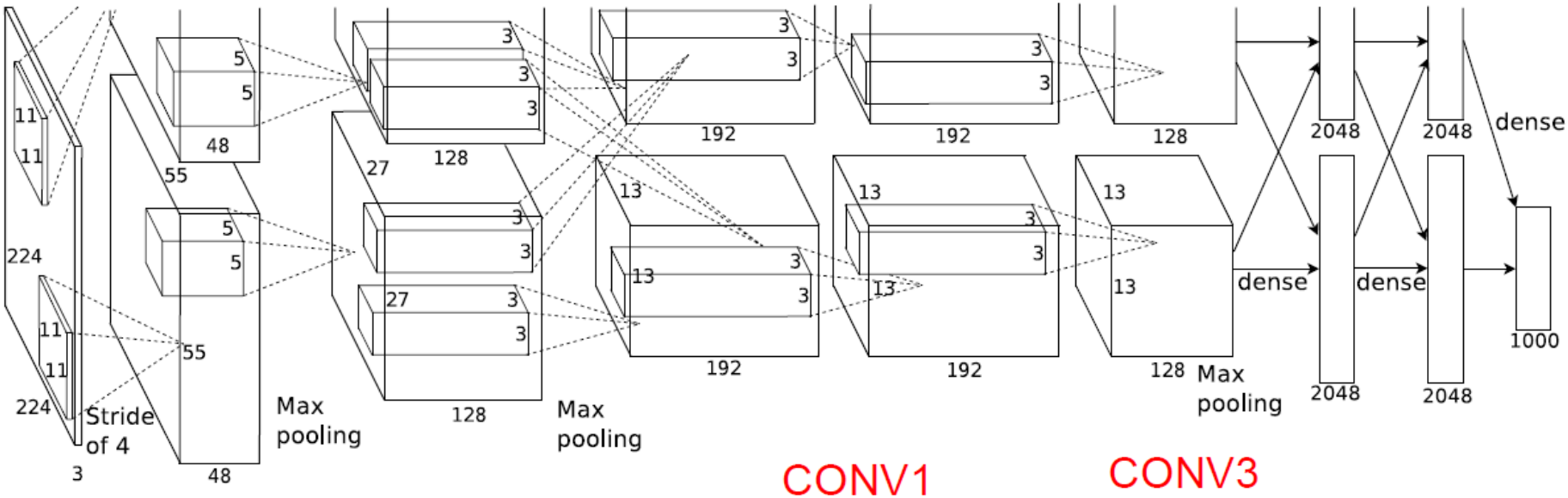
- **评价指标**
  - 精确率 Precision = TP/(TP + FP)
  - 召回率　　Recall = TP/(TP + FN)
  - PR曲线：Precision-Recall曲线
  - AP: PR 曲线下的面积(AUC, Area Under Curve)
  - mAP(mean Average Precision)



7.2 基于卷积神经网络的图像分类



1）AlexNet



CONV1
MAX POOL1
NORM1
CONV2
MAX POOL2
NORM2

CONV3
CONV4
CONV5
Max POOL3
FC6
FC7
FC8

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky et al. 2012]

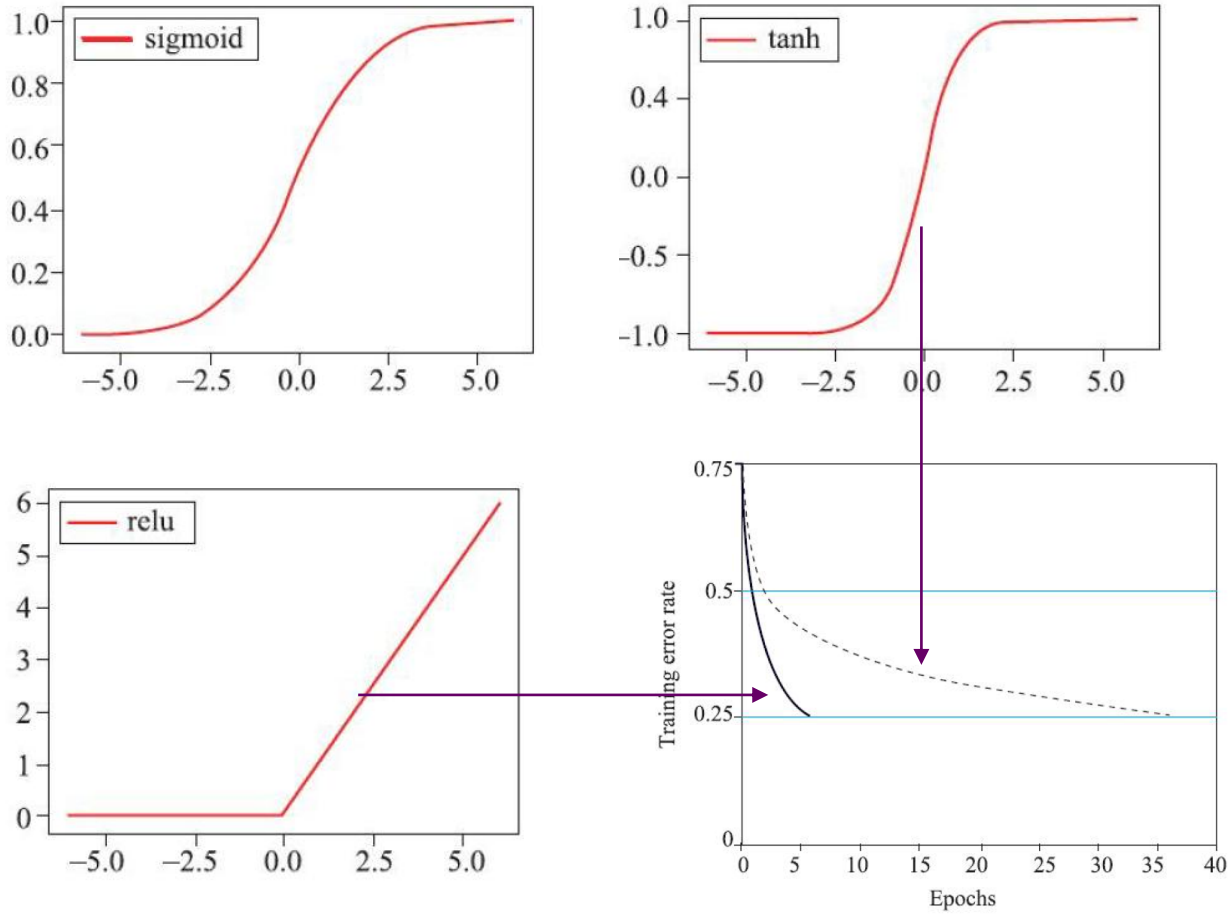
1）AlexNet

$$W' = (W - F + 2P) / S + 1$$

input	layer	kernel	kernel_num	stride	pad	output	parameters
227*227*3	CONV1	11*11*3	96	4	0	55*55*96	(11*11*3)*96 = 35K
55*55*96	MAX POOL1	3*3		2	0	27*27*96	
27*27*96	NORM1						
27*27*96	CONV2	5*5*96	256	1	2	27*27*256	(5*5*96)*256 = 614K
27*27*256	MAX POOL2	3*3		2	0	13*13*256	
13*13*256	NORM2						
13*13*256	CONV3	3*3*256	384	1	1	13*13*384	(3*3*256)*384 = 885K
13*13*384	CONV4	3*3*384	384	1	1	13*13*384	(3*3*384)*384 = 1327K
13*13*384	CONV5	3*3*384	256	1	1	13*13*256	(3*3*384)*256 = 885K
13*13*256	Max POOL3	3*3		2	0	6*6*256	
6*6*256	FC6					4096	(6*6*256)*4096 =3775W
4096	FC7					4096	4096*4096 = 1678W
4096	FC8					1000	4096* 1000 = 410W

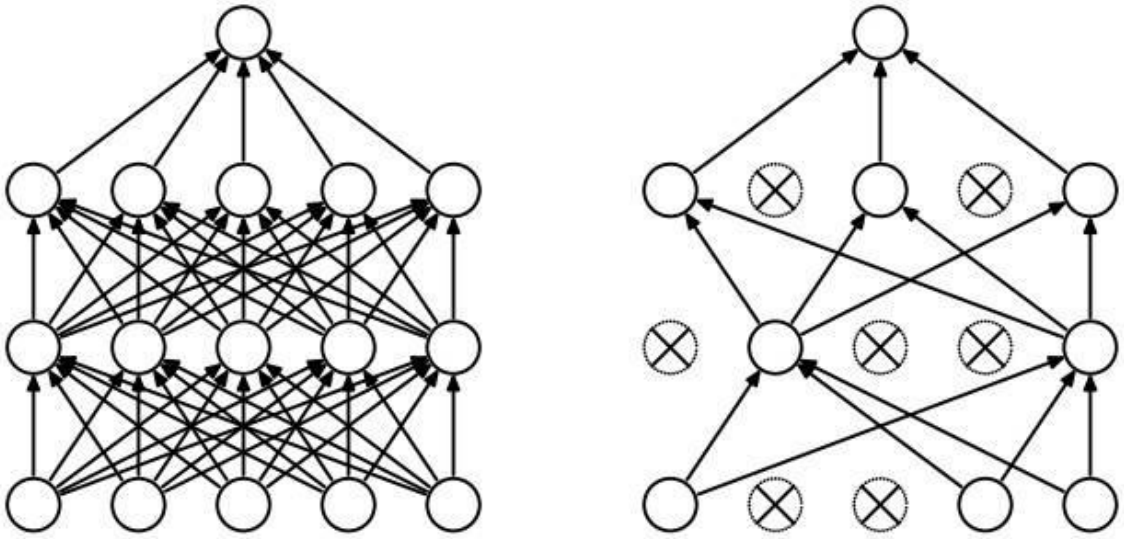
1）AlexNet

- 使用ReLu作为激活函数：从sigmoid/tanh到Relu



1）AlexNet

- 使用ReLu作为激活函数：从sigmoid/tanh到Relu
- 过拟合处理
  - 数据增强：随机裁剪、图像翻转、平移、缩放、旋转
  - 使用dropout：网络训练时，以一定概率随机dropout(丢弃)一些被激活的神经元，使得模型更鲁棒



(a) Standard Neural Net　　(b) After applying dropout.

网络模型

```
class AlexNet(nn.Module):
    def __init__(self, num_classes=1000, init_weights=False):
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(# 模块打包
            nn.Conv2d(3, 48, kernel_size=11, stride=4, padding=2),  # input[3, 224, 224]  output[48, 55, 55]
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),                  # output[48, 27, 27]
            nn.Conv2d(48, 128, kernel_size=5, padding=2),           # output[128, 27, 27]
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),                  # output[128, 13, 13]
            nn.Conv2d(128, 192, kernel_size=3, padding=1),          # output[192, 13, 13]
            nn.ReLU(inplace=True),
            nn.Conv2d(192, 192, kernel_size=3, padding=1),          # output[192, 13, 13]
            nn.ReLU(inplace=True),
            nn.Conv2d(192, 128, kernel_size=3, padding=1),          # output[128, 13, 13]
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),                  # output[128, 6, 6]
        )
        self.classifier = nn.Sequential(
            nn.Dropout(p=0.5),
            nn.Linear(128 * 6 * 6, 2048),
            nn.ReLU(inplace=True),
            nn.Dropout(p=0.5),
            nn.Linear(2048, 2048),
            nn.ReLU(inplace=True),
            nn.Linear(2048, num_classes),
        )
```

网络模型：特征提取

```
self.features = nn.Sequential(# 模块打包
    nn.Conv2d(3, 48, kernel_size=11, stride=4, padding=2),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Conv2d(48, 128, kernel_size=5, padding=2),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=3, stride=2),
    ......
```

input	layer	kernel	kernel_num	stride	pad	output
224*224*3	CONV1	11*11*3	96 /2	4	2	55*55*96
55*55*96	MAX POOL1	3*3		2	0	27*27*96
27*27*96	NORM1					
27*27*96	CONV2	5*5*96	256 /2	1	2	27*27*256
27*27*256	MAX POOL2	3*3		2	0	13*13*256
13*13*256	NORM2					
13*13*256	CONV3	3*3*256	384 /2	1	1	13*13*384
13*13*384	CONV4	3*3*384	384 /2	1	1	13*13*384
13*13*384	CONV5	3*3*384	256 /2	1	1	13*13*256

网络模型：分类+前向计算

```
self.classifier = nn.Sequential(
    nn.Dropout(p=0.5),
    nn.Linear(128 * 6 * 6, 2048),
    nn.ReLU(inplace=True),
    nn.Dropout(p=0.5),
    nn.Linear(2048, 2048),
    nn.ReLU(inplace=True),
    nn.Linear(2048, num_classes),
```

6*6*256 /2	FC6					4096
4096 /2	FC7					4096
4096 /2	FC8					1000

```
def forward(self, x):
    x = self.features(x)
    x = torch.flatten(x, start_dim=1)
    x = self.classifier(x)
    return x
```

网络训练

```python
from model import AlexNet


def main():
    device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")  # 选择设备
    print("using {} device.".format(device))

    # 训练集和验证集的图像预处理
    data_transform = {
        "train": transforms.Compose([transforms.RandomResizedCrop(224),  # 随机裁剪
                                     transforms.RandomHorizontalFlip(),  # 水平随机翻转
                                     transforms.ToTensor(),
                                     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))]),
        "val": transforms.Compose([transforms.Resize((224, 224)),  # cannot 224, must (224, 224)
                                   transforms.ToTensor(),
                                   transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])}

    data_root = os.path.abspath(os.path.join(os.getcwd(), "../.."))  # get data root path
    image_path = os.path.join(data_root, "data_set", "flower_data")  # flower data set path
    assert os.path.exists(image_path), "{} path does not exist.".format(image_path)
    train_dataset = datasets.ImageFolder(root=os.path.join(image_path, "train"),
                                         transform=data_transform["train"])  # 使用train的处理方法
```

网络训练

```
batch_size = 32
nw = min([os.cpu_count(), batch_size if batch_size > 1 else 0, 8])  # number of workers
print('Using {} dataloader workers every process'.format(nw))

train_loader = torch.utils.data.DataLoader(train_dataset,
                                           batch_size=batch_size, shuffle=True,
                                           num_workers=nw)

validate_dataset = datasets.ImageFolder(root=os.path.join(image_path, "val"),
                                        transform=data_transform["val"])
val_num = len(validate_dataset)  # 验证集数据量
validate_loader = torch.utils.data.DataLoader(validate_dataset,
                                              batch_size=4, shuffle=False,
                                              num_workers=nw)
```

超参数

```python
net = AlexNet(num_classes=5, init_weights=True)
# pata = list(net.parameters()) # 查看模型参数

net.to(device)  # 设备指定
loss_function = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(), lr=0.0002)

epochs = 10
save_path = './AlexNet.pth'
best_acc = 0.0
train_steps = len(train_loader)  # 训练一轮所需要的步数，等于train_num/batch_size
for epoch in range(epochs):
    # train
    net.train()     # 使用dropout
    running_loss = 0.0
    train_bar = tqdm(train_loader, file=sys.stdout)  # 进度条
    for step, data in enumerate(train_bar):
        images, labels = data
①       optimizer.zero_grad()  # 清空梯度

        # forward + backward + optimize
②       outputs = net(images.to(device))
        loss = loss_function(outputs, labels.to(device))
③       loss.backward()
④       optimizer.step()  # 更新参数

        # print statistics
        running_loss += loss.item()
        train_bar.desc = "train epoch[{}/{}] loss:{:.3f}".format(epoch + 1,
                                                                 epochs,
                                                                 loss)
```

超参数

```python
net = AlexNet(num_classes=5, init_weights=True)
# pata = list(net.parameters()) # 查看模型参数

net.to(device)  # 设备指定
loss_function = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(), lr=0.0002)

epochs = 10
save_path = './AlexNet.pth'
best_acc = 0.0
train_steps = len(train_loader)  # 训练一轮所需要的步数,
for epoch in range(epochs):
    # train
    net.train()     # 使用dropout
    running_loss = 0.0
    train_bar = tqdm(train_loader, file=sys.stdout)
    for step, data in enumerate(train_bar):
        images, labels = data
①       optimizer.zero_grad()  # 清空梯度

        # forward + backward + optimize
②       outputs = net(images.to(device))
        loss = loss_function(outputs, labels.to(device))
③       loss.backward()
④       optimizer.step()  # 更新参数

        # print statistics
        running_loss += loss.item()
        train_bar.desc = "train epoch[{}/{}] loss:{:.3f}".format(epoch + 1,
                                                                 epochs,
                                                                 loss)
```

```python
# validate
net.eval()  # 关闭dropout
acc = 0.0  # accumulate accurate number / epoch
with torch.no_grad():
    val_bar = tqdm(validate_loader, file=sys.stdout)
    for val_data in val_bar:
        val_images, val_labels = val_data
        outputs = net(val_images.to(device))
        # 一个batch的预测结果，dim=1在维度1上取最大值，[1]返回对应index
        predict_y = torch.max(outputs, dim=1)[1]
        acc += torch.eq(predict_y, val_labels.to(device)).sum().item()

# 到此验证集全部处理完，计算验证集准确率
val_accurate = acc / val_num
print('[epoch %d] train_loss: %.3f  val_accuracy: %.3f' %
      (epoch + 1, running_loss / train_steps, val_accurate))

# 完成一次epoch的训练+验证，如果在验证集准确率提高，则保留网络参数
if val_accurate > best_acc:
    best_acc = val_accurate
    torch.save(net.state_dict(), save_path)
```

超参数

预测/推理

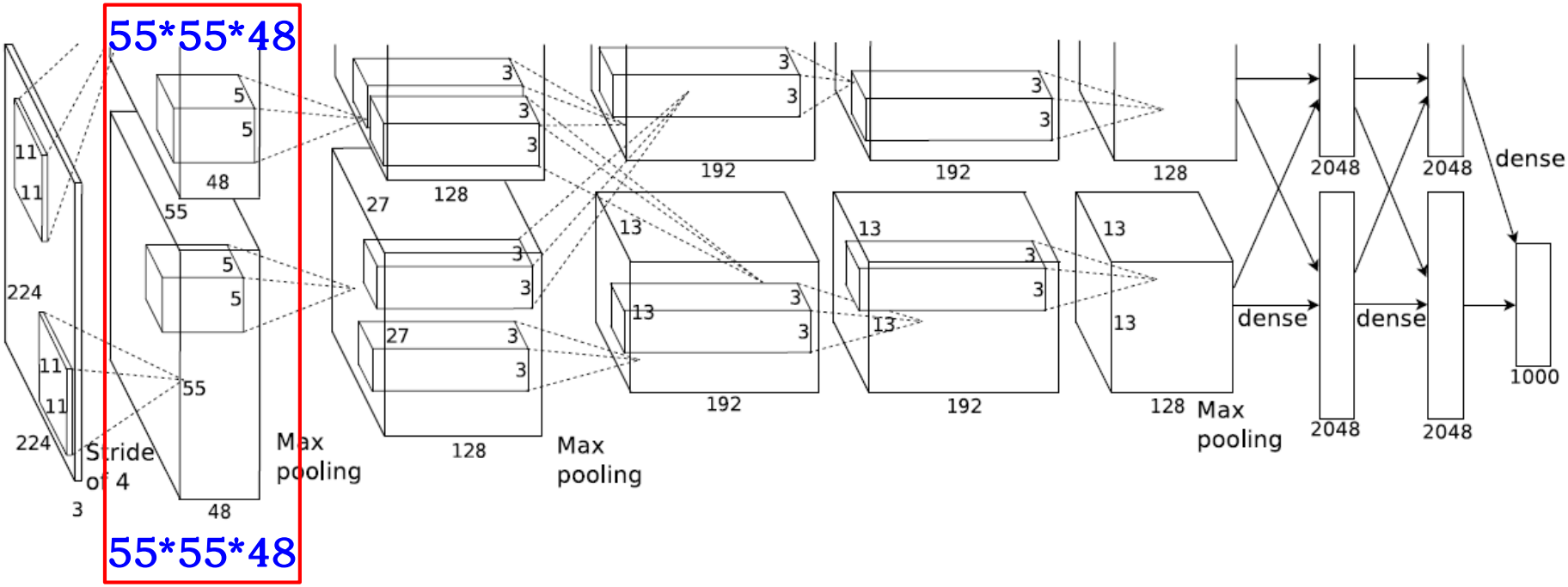
```python
def main():
    device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

    data_transform = transforms.Compose(
        [transforms.Resize((224, 224)),
         transforms.ToTensor(),
         transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

    # load image
    img_path = "../tulip.jpg"
    assert os.path.exists(img_path), "file: '{}' dose not exist.".format(img_path)
    img = Image.open(img_path)

    plt.imshow(img)
    # [N, C, H, W]
    img = data_transform(img)
    # expand batch dimension
    img = torch.unsqueeze(img, dim=0)  # 加上batch的维度，和后面的squeeze对应
```

预测/推理

```
# create model
model = AlexNet(num_classes=5).to(device)

# load model weights
weights_path = "./AlexNet.pth"
assert os.path.exists(weights_path), "file: '{}' dose not exist.".format(weights_path)
model.load_state_dict(torch.load(weights_path))

model.eval()
with torch.no_grad():
    # predict class
    output = torch.squeeze(model(img.to(device))).cpu()  # 输出压缩掉batch的维度，得到一张图的预测结果，共num_classes维
    predict = torch.softmax(output, dim=0)      # softmax处理后，得到每个类别的概率
    predict_cla = torch.argmax(predict).numpy() # 得到类别index

print_res = "class: {}   prob: {:.3}".format(class_indict[str(predict_cla)],
                                             predict[predict_cla].numpy())
plt.title(print_res)        # 预测的类别及对应概率作为图像的标题

for i in range(len(predict)):  # 打印所有的类别和对应的概率
    print("class: {:10}   prob: {:.3}".format(class_indict[str(i)],
                                              predict[i].numpy()))
plt.show()
```

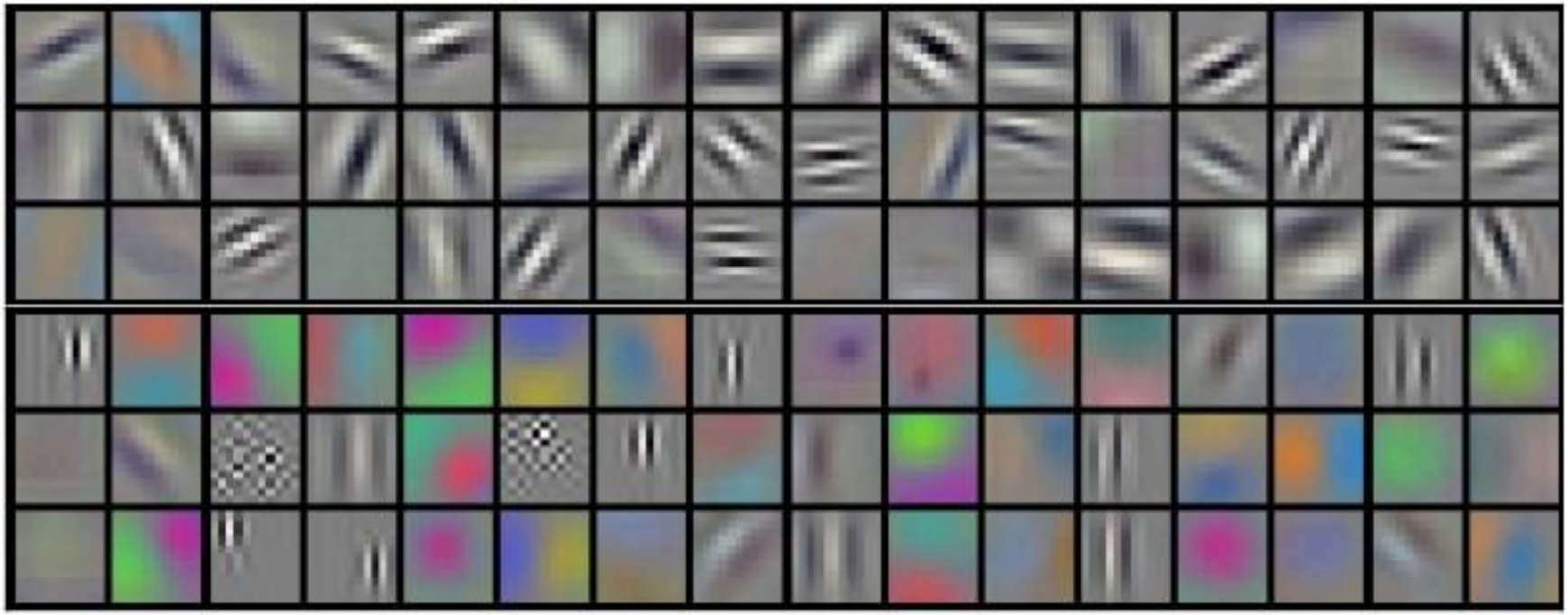
1）AlexNet

- 使用ReLu作为激活函数：从sigmoid/tanh到Relu
- 过拟合处理
- 使用GPU并行

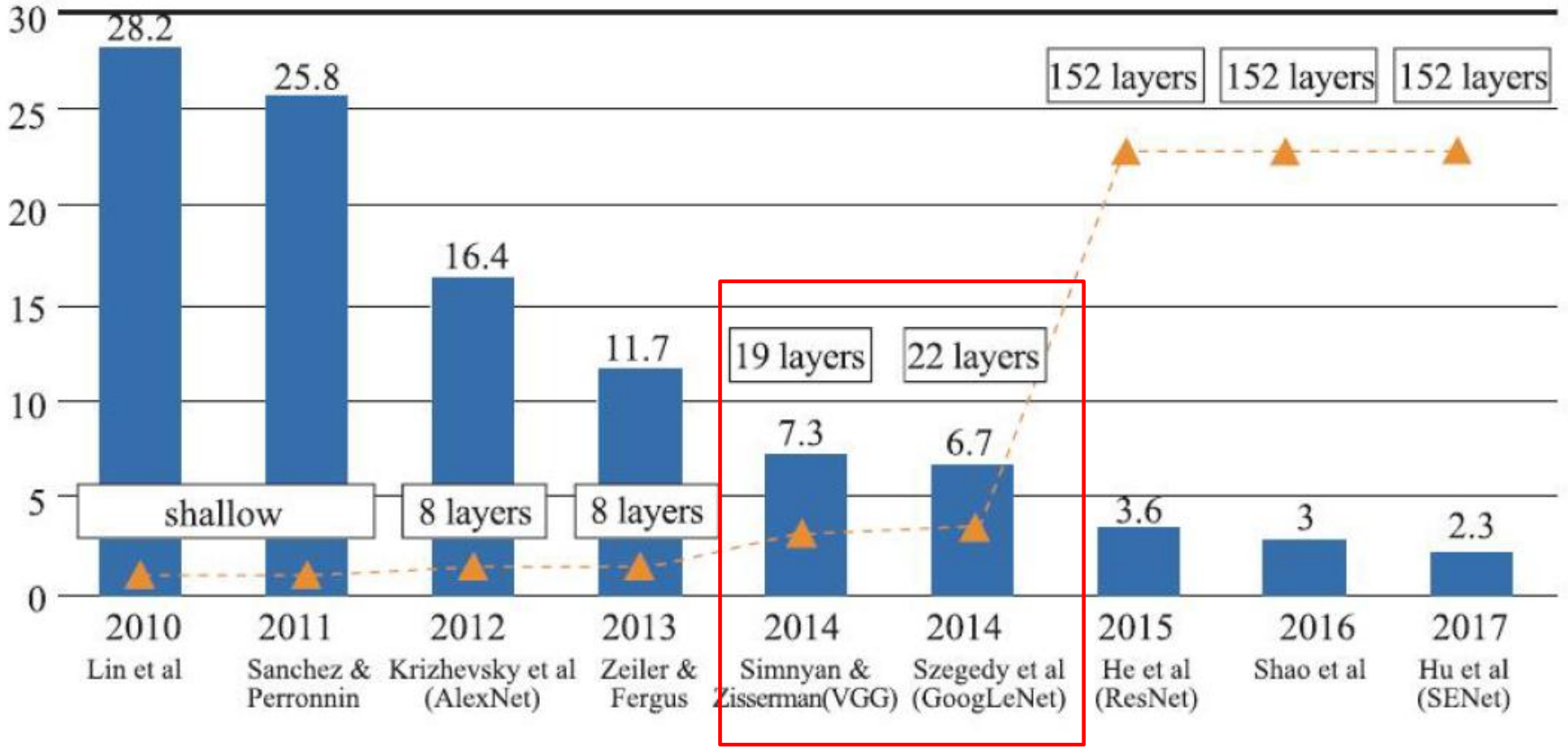


1）AlexNet

- Conv1的96个11×11×3卷积核from AlexNet

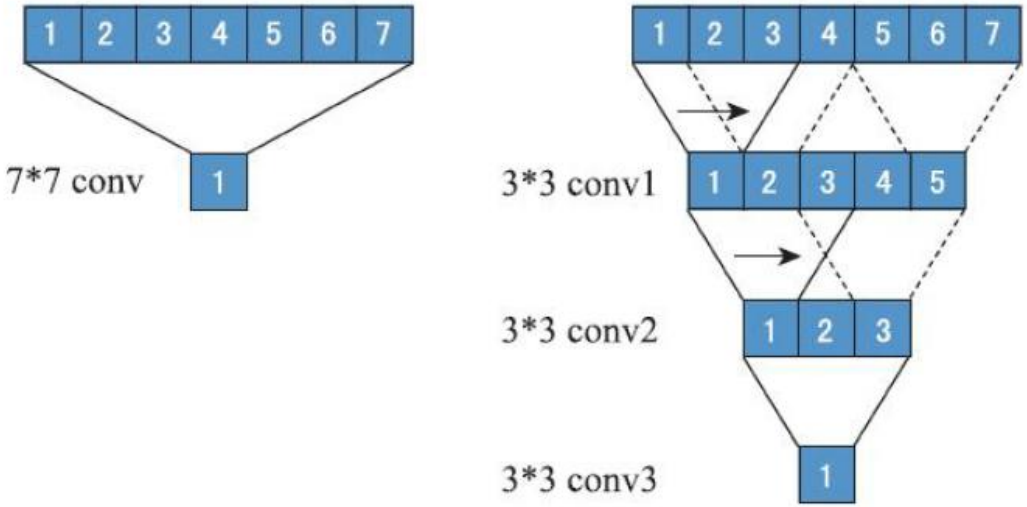


7.2 基于卷积神经网络的图像分类

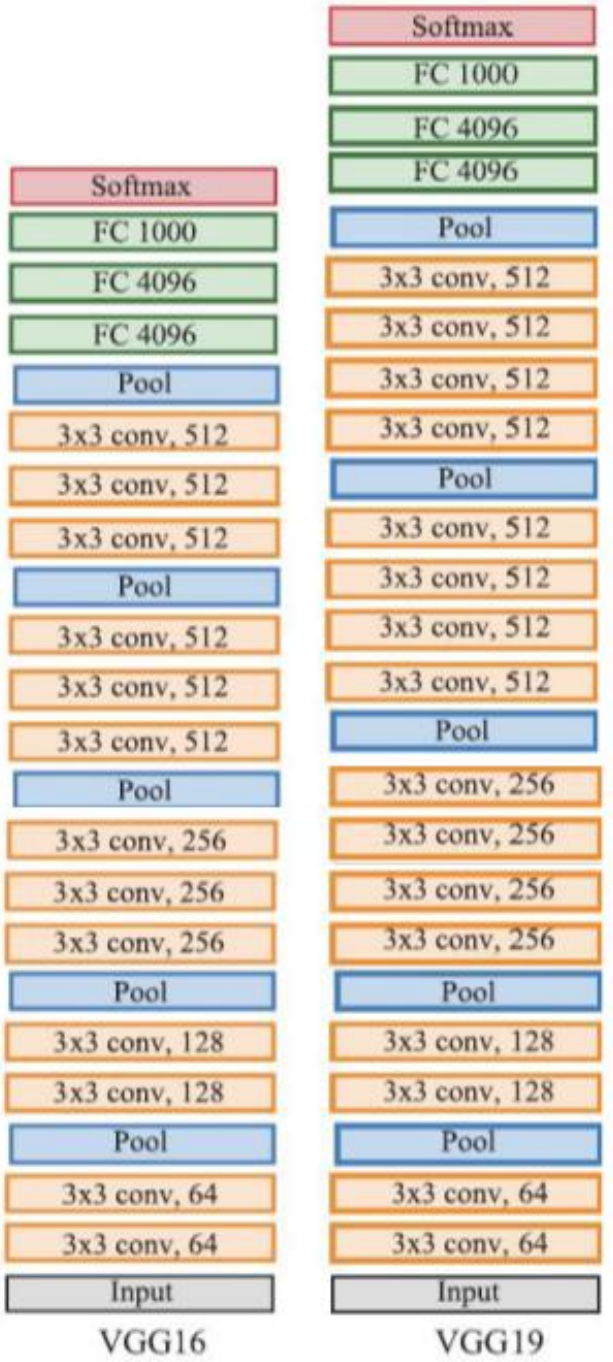


2） VGGNet

- VGGNet：只使用3*3 kernel，3组3*3的kernel与1个7*7的kernel产生相同的效果



VGGNet-Very Deep Convolutional Networks for Large-Scale Image Recognition [Simonyan and Zisserman 2014]



VGG16 网络结构

memory:
24M * 4 bytes
≈96MB/image

params: 138M

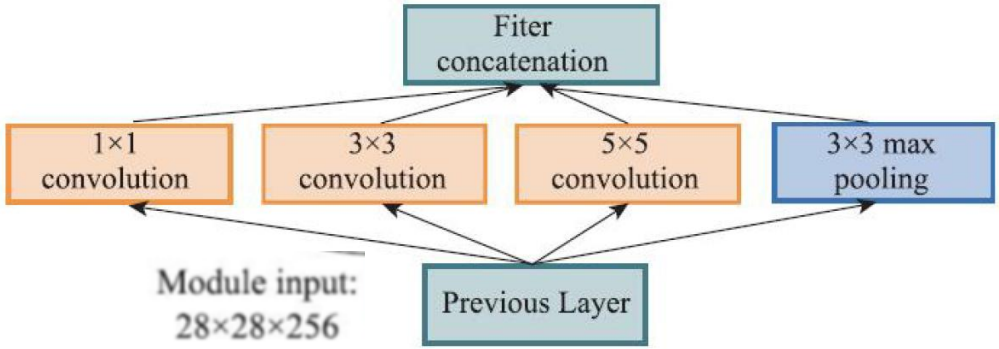
input_size	layer	memory	parameters
[224x224x3]	input	224*224*3=150K	0
[224x224x64]	CONV3-64	224*224*64=3.2M	(3*3*3)*64 = 1,728
[224x224x64]	CONV3-64	224*224*64=3.2M	(3*3*64)*64 = 36,864
[112x112x64]	POOL2	112*112*64=800K	0
[112x112x128]	CONV3-128	112*112*128=1.6M	(3*3*64)*128 = 73,728
[112x112x128]	CONV3-128	112*112*128=1.6M	(3*3*128)*128 = 147,456
[56x56x128]	POOL2	56*56*128=400K	0
[56x56x256]	CONV3-256	56*56*256=800K	(3*3*128)*256 = 294,912
[56x56x256]	CONV3-256	56*56*256=800K	(3*3*256)*256 = 589,824
[56x56x256]	CONV3-256	56*56*256=800K	(3*3*256)*256 = 589,824
[28x28x256]	POOL2	28*28*256=200K	0
[28x28x512]	CONV3-512	28*28*512=400K	(3*3*256)*512 = 1,179,648
[28x28x512]	CONV3-512	28*28*512=400K	(3*3*512)*512 = 2,359,296
[28x28x512]	CONV3-512	28*28*512=400K	(3*3*512)*512 = 2,359,296
[14x14x512]	POOL2	14*14*512=100K	0
[14x14x512]	CONV3-512	14*14*512=100K	(3*3*512)*512 = 2,359,296
[14x14x512]	CONV3-512	14*14*512=100K	(3*3*512)*512 = 2,359,296
[14x14x512]	CONV3-512	14*14*512=100K	(3*3*512)*512 = 2,359,296
[7x7x512]	POOL2	7*7*512=25K	0
[1x1x4096]	FC	4096	7*7*512*4096 = 102,760,448
[1x1x4096]	FC	4096	4096*4096 = 16,777,216
[1x1x1000]	FC	1000	4096*1000 = 4,096,000

VGG多种结构

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

3) GoogLeNet

- GoogLeNet：通过Inception结构，构建稀疏的、高计算性能的网络结构

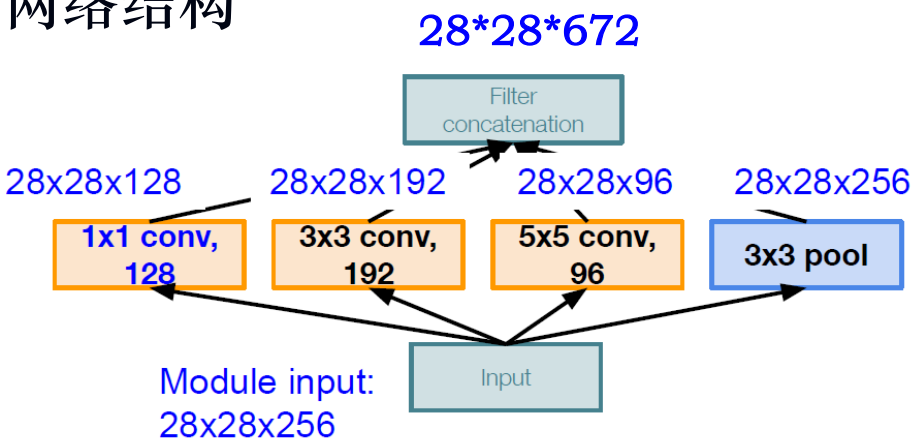


input	layer	output_size	computation
28*28*256	1*1*128 conv	28*28*128	28*28*128*1*1*256
	3*3*192 conv	28*28*192	28*28*192*3*3*256
	5*5*96 conv	28*28*96	28*28*96*5*5*256
	3*3 pool	28*28*256	
		28*28*672	854M

Going deeper with convolutions [Szegedy et al. 2014]

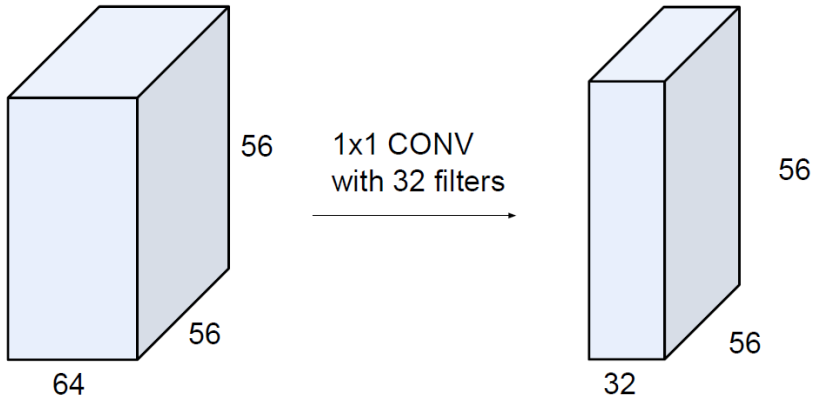
3) GoogLeNet

- GoogLeNet：通过Inception结构，构建稀疏的、高计算性能的网络结构



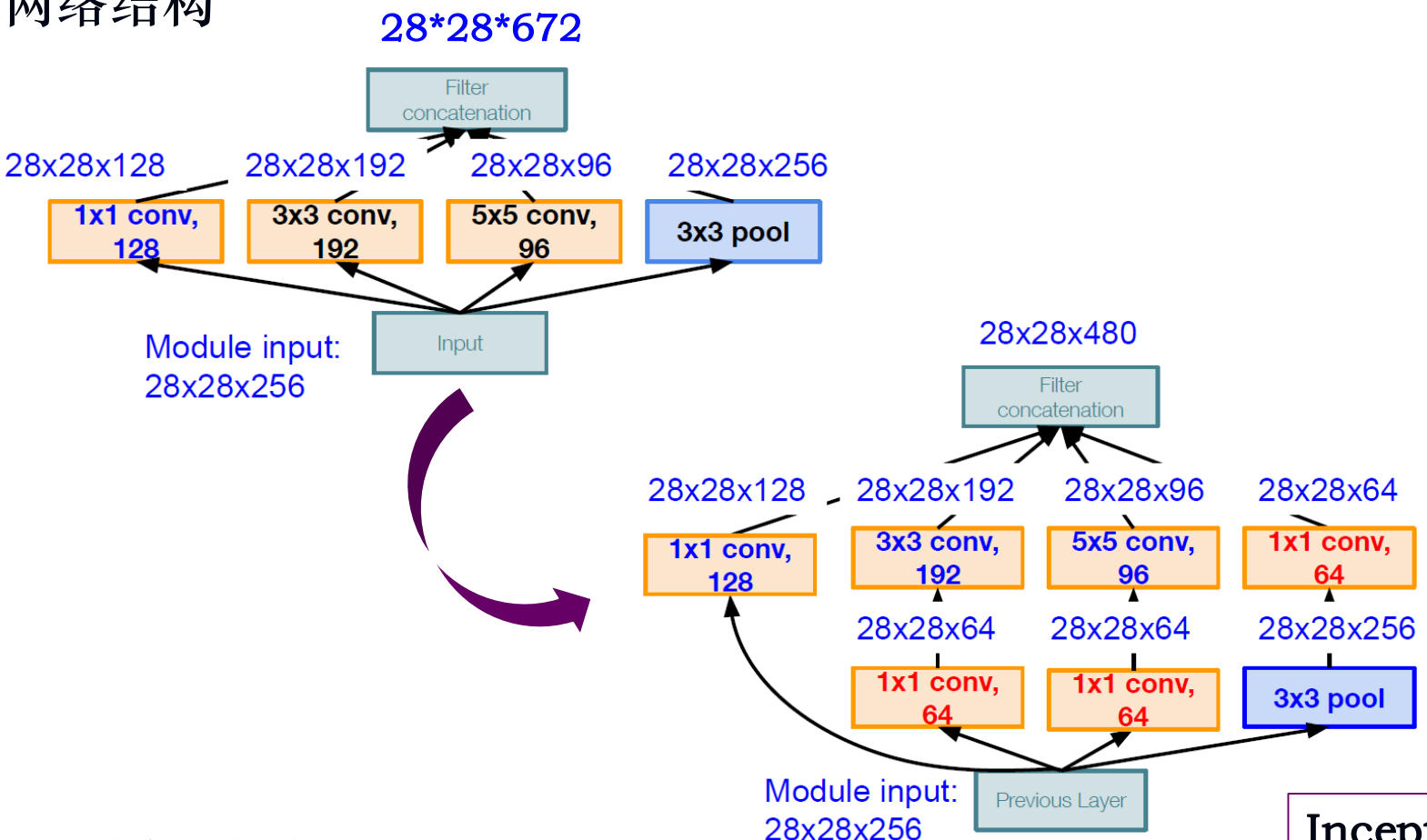
output_size	computation
28*28*128	28*28*128*1*1*256
28*28*192	28*28*192*3*3*256
28*28*96	28*28*96*5*5*256
28*28*256	

28*28*672　　854M



3) GoogLeNet

- GoogLeNet：通过Inception结构，构建稀疏的、高计算性能的网络结构



Inception v1

3) GoogLeNet

- GoogLeNet：通过Inception结构，构建稀疏的、高计算性能的网络结构

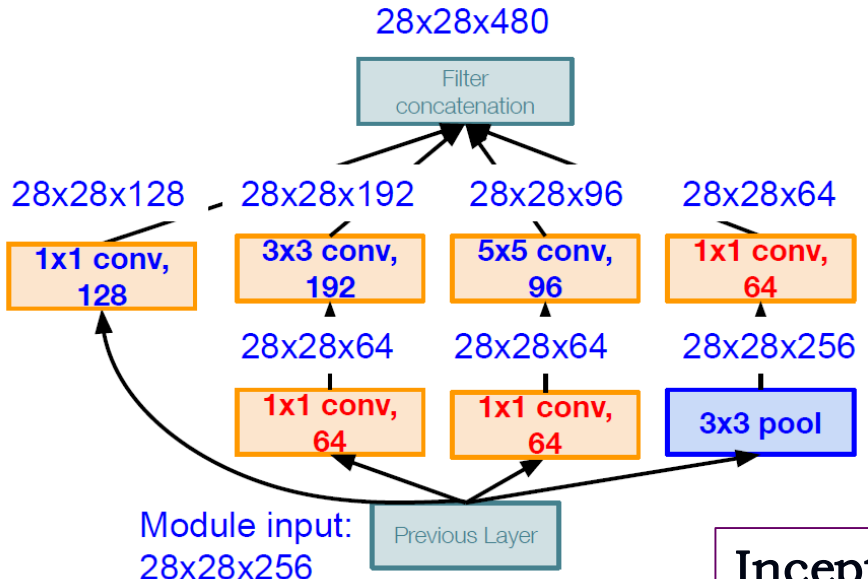
layer	computation
1*1*128 conv	28*28*128*1*1*256
3*3*192 conv	28*28*192*3*3*256
5*5*96 conv	28*28*96*5*5*256

854M

[1×1 conv, 64] 28×28×64×1×1×256
[1×1 conv, 64] 28×28×64×1×1×256
[1×1 conv, 128] 28×28×128×1×1×256
[3×3 conv, 192] 28×28×192×3×3×64
[5×5 conv, 96] 28×28×96×5×5×64
[1×1 conv, 64] 28×28×64×1×1×256

358M

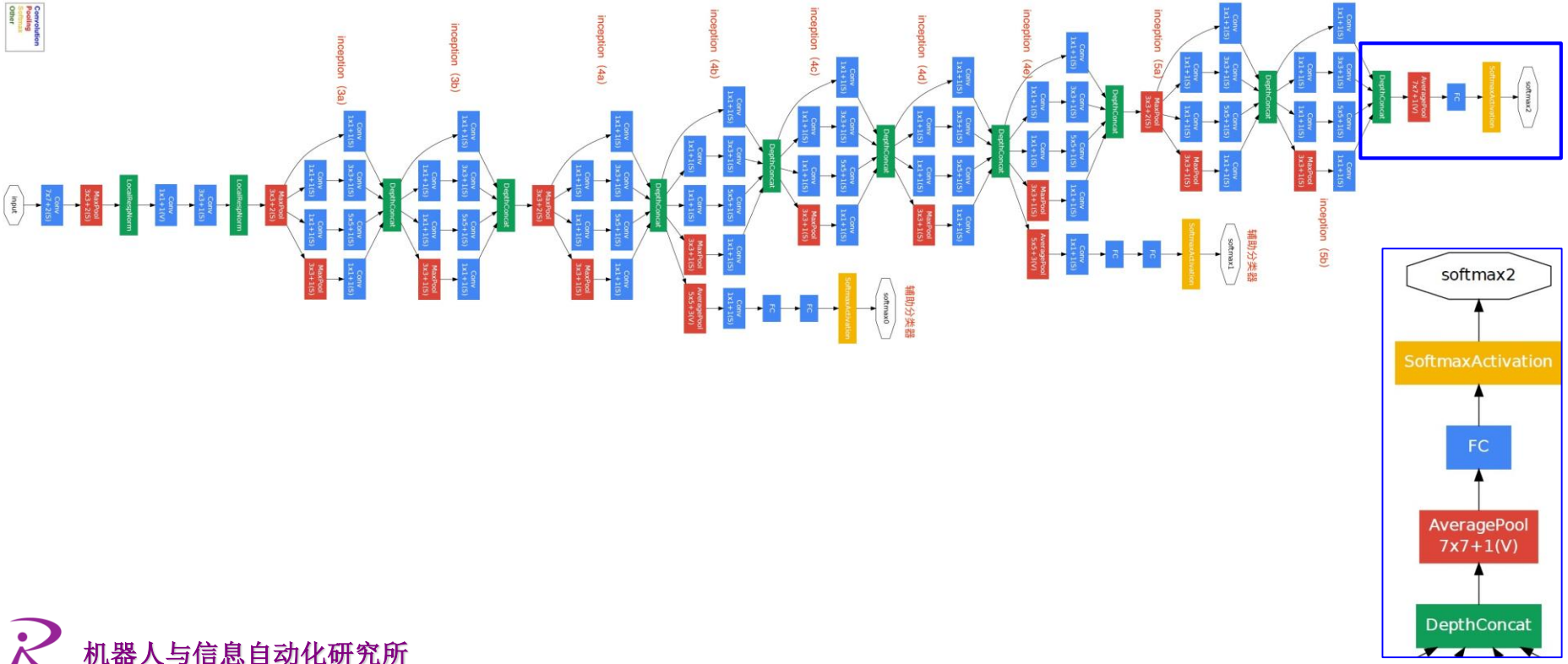
params: 5M
AlexNet的1/12
VGG-16的1/27



Inception v1

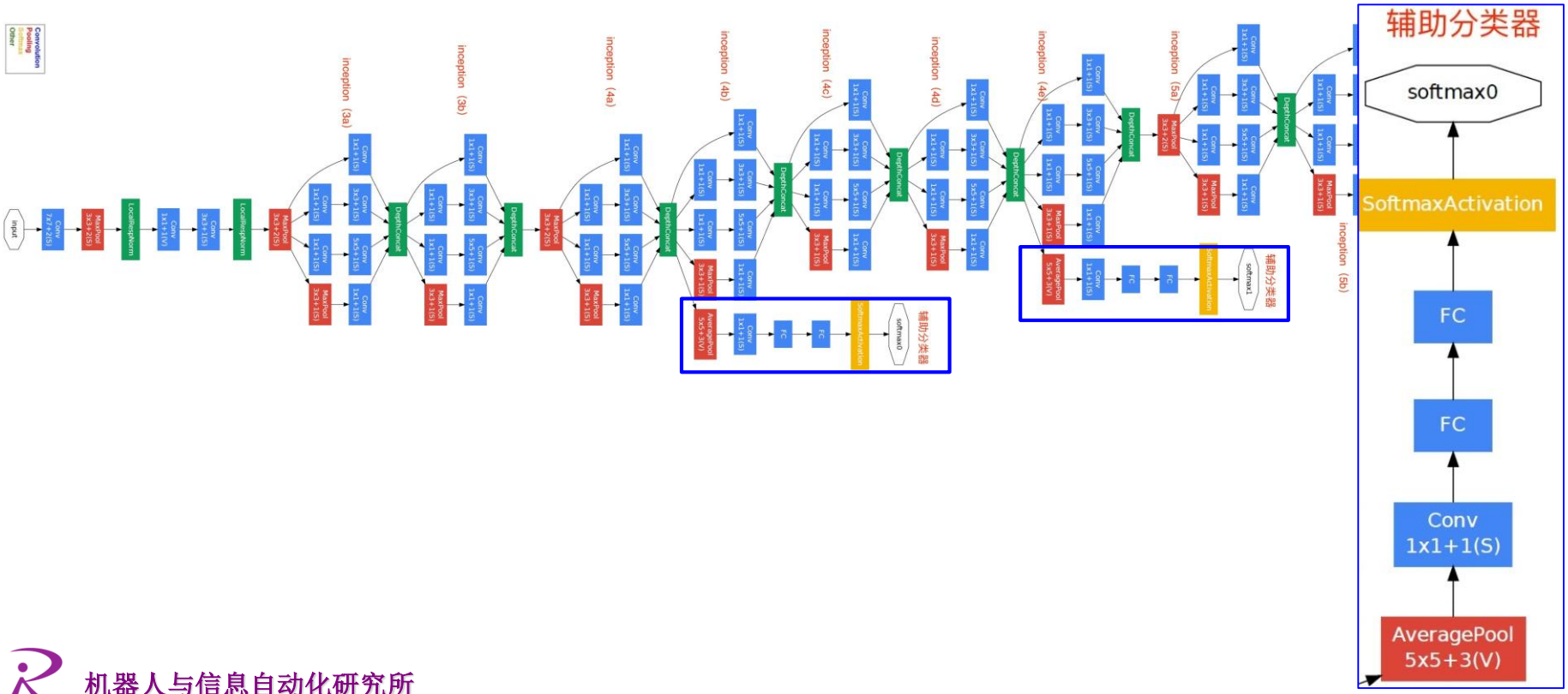
3）GoogLeNet

- GoogLeNet：通过Inception结构，构建稀疏的、高计算性能的网络结构
- 传统结构+Inception结构+分类器（一个全连接层）



3）GoogLeNet

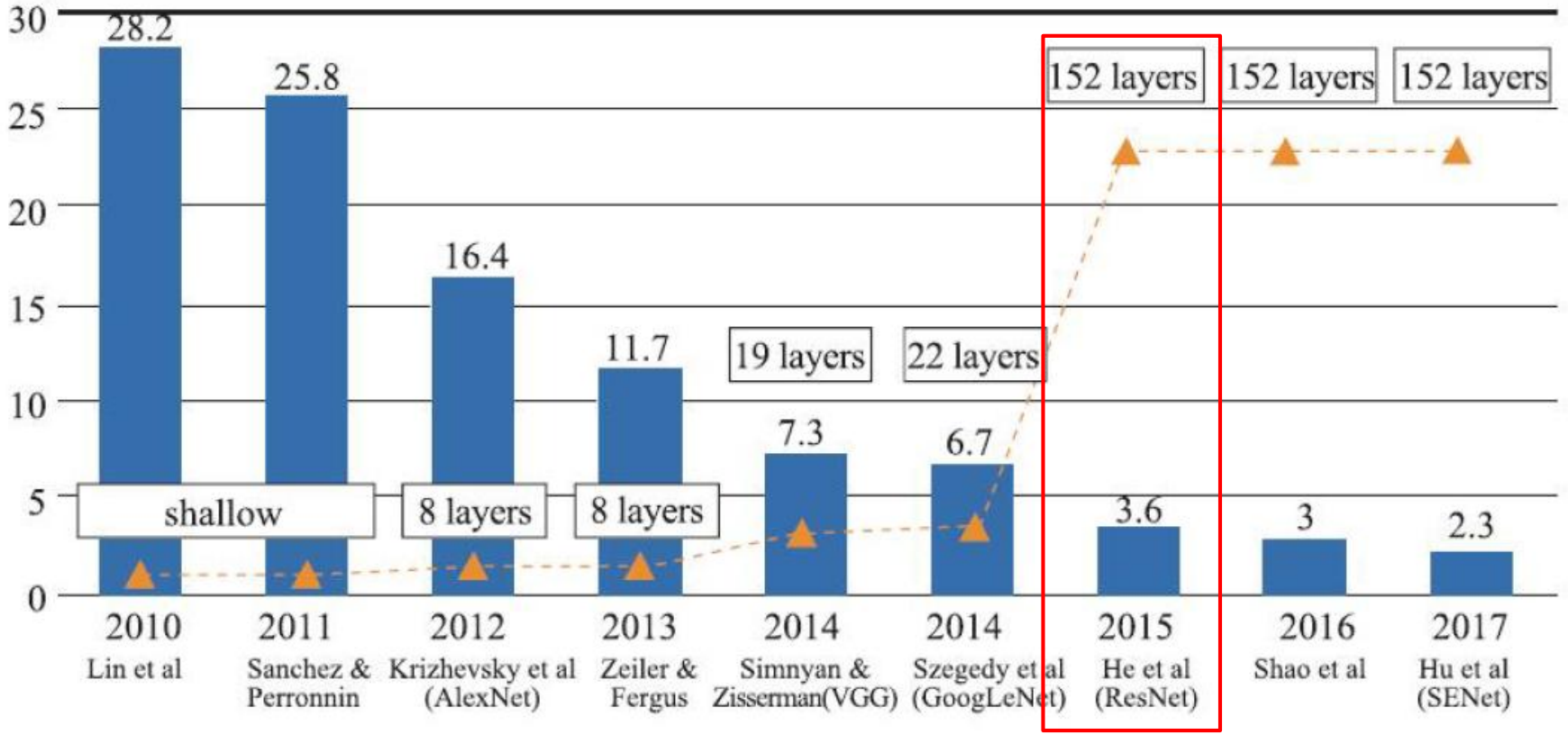
- GoogLeNet：通过Inception结构，构建稀疏的、高计算性能的网络结构
- 传统结构+Inception结构+分类器（一个全连接层）



3) GoogLeNet

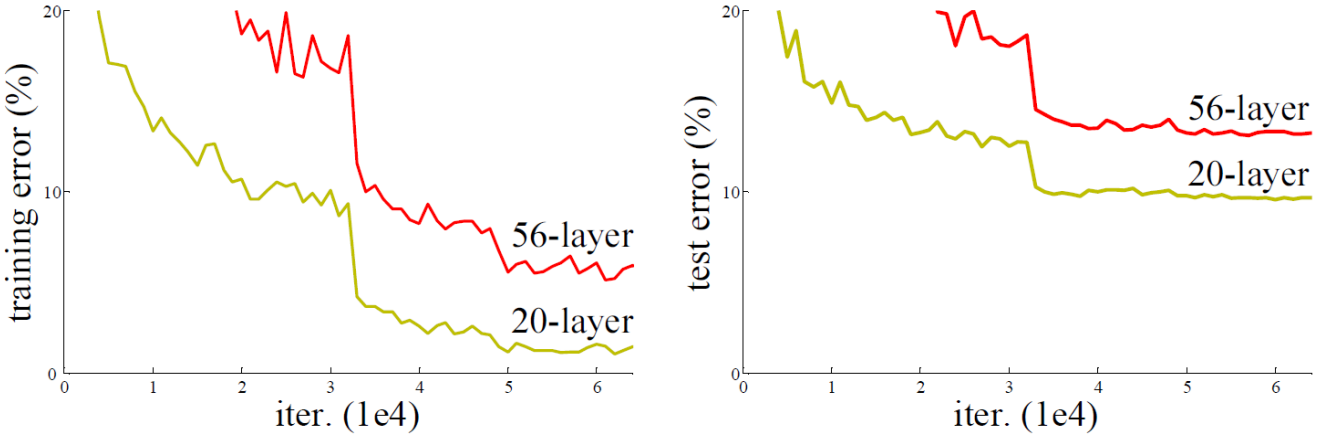
type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

7.3 基于卷积神经网络的图像分类



4) ResNet

- *Is learning better networks as easy as stacking more layers?*
- 网络退化（degradation）

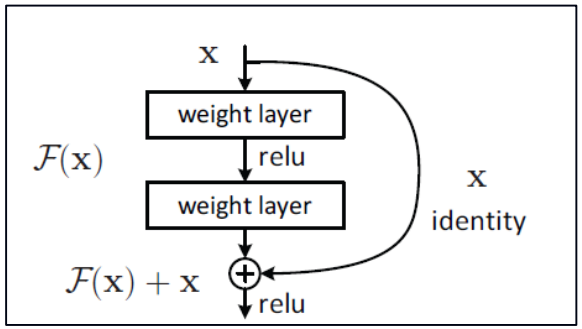


- 恒等映射(Identity Mapping)
  - 把低层的特征传到高层，效果应该至少不比浅层的网络效果差
  - 在网络高层和低层之间加入恒等映射来达到此效果

Deep Residual Learning for Image Recognition [He et al. 2015]

4) ResNet

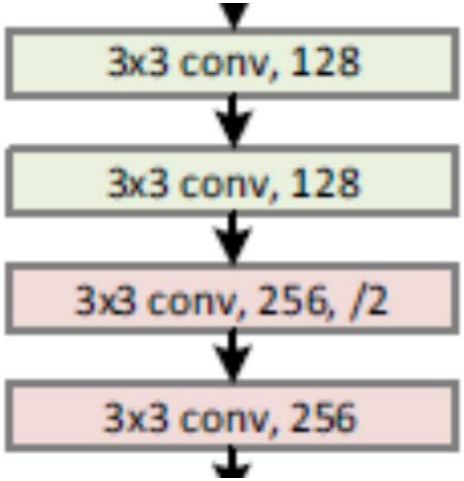
- **残差网络**
  - $F(x)+x$可以通过在前馈网络中加入“快捷连接”(shortcut connections)来实现

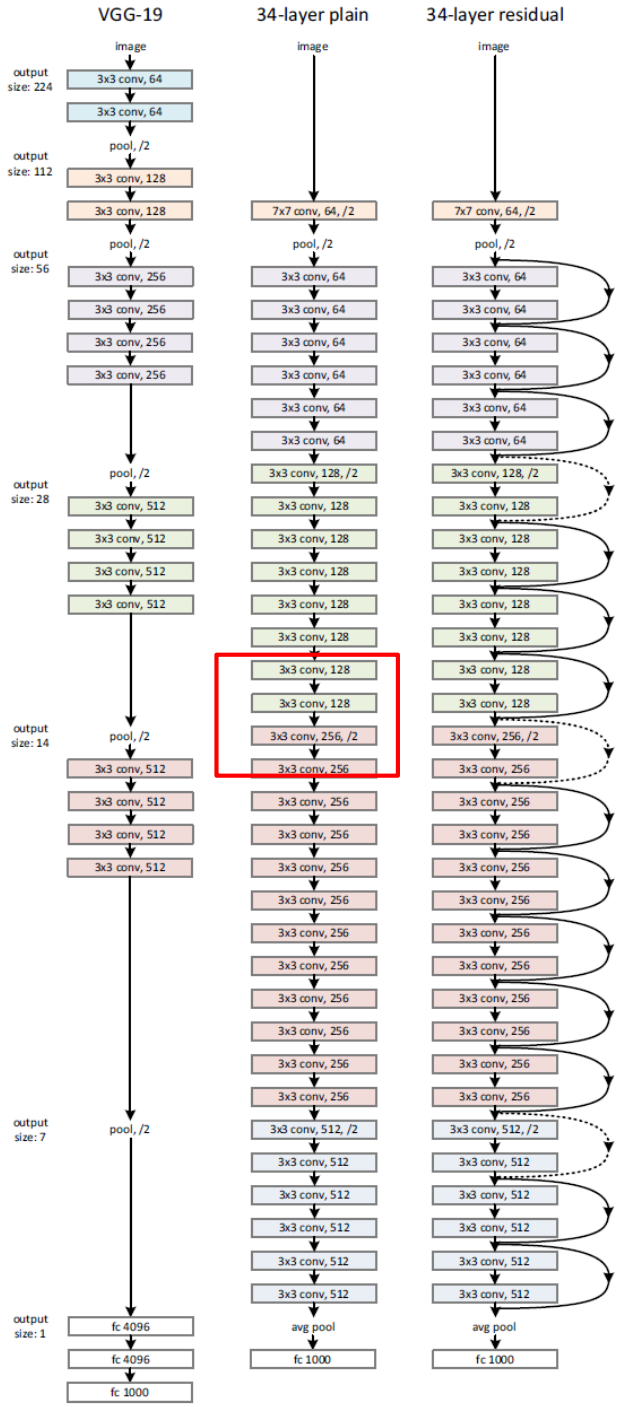



```
def forward(self, x):
    identity = x    # shortcut分支

    # 主分支
    out = self.conv1(x)
    out = self.bn1(out)
    out = self.relu(out)
    out = self.conv2(out)
    out = self.bn2(out)

    # 注意此处没有激活，加上shortcut后，再进行激活
    out += identity
    out = self.relu(out)

    return out
```

4）ResNet

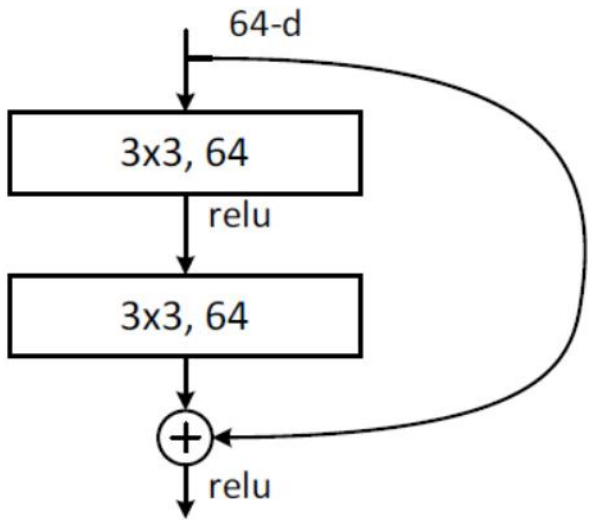
- 网络结构(Plain Network)
  - 对于相同尺寸的输出特征图，每层必须含有相同数量的卷积核
  - 如果特征图的尺寸减半，则卷积核数量翻倍，以保持每层的时间复杂度。
  - 一个全连接层

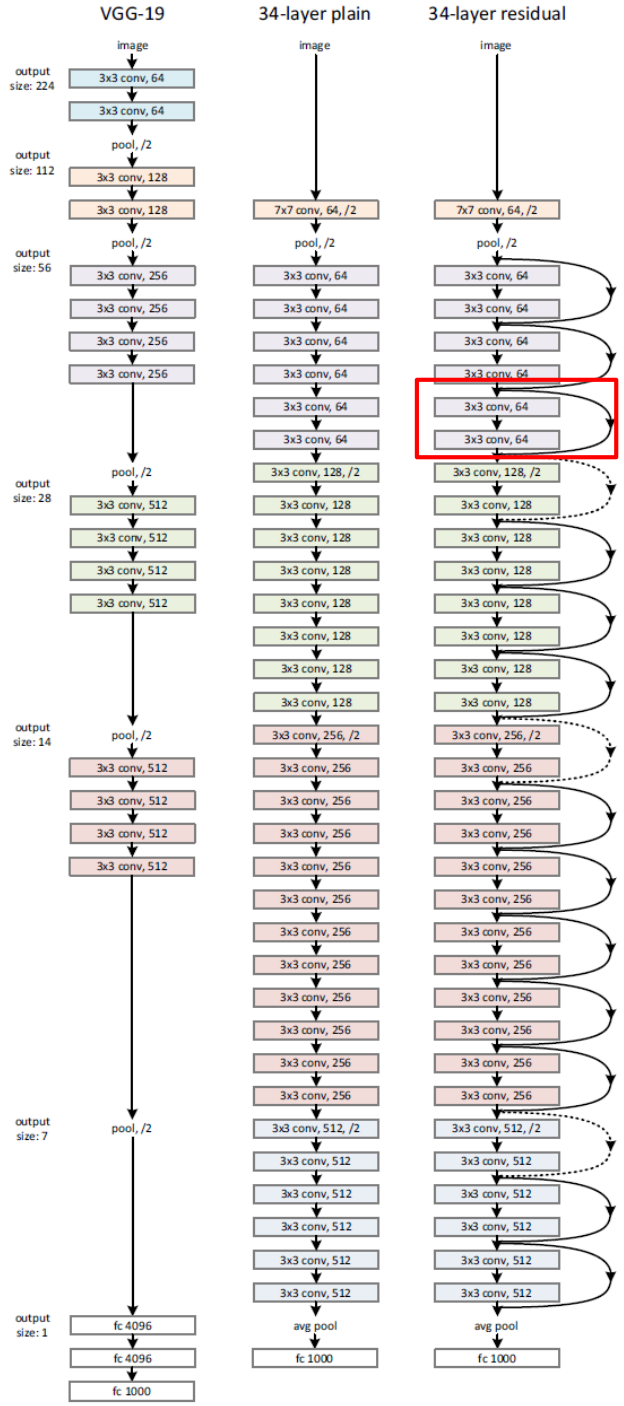




4）ResNet

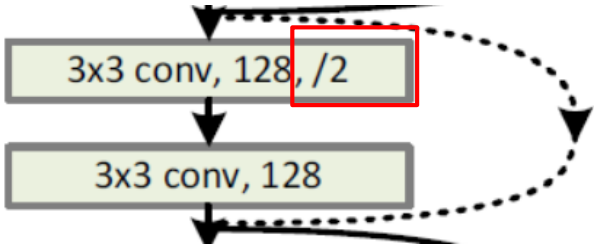
- 网络结构(ResNet18/34)
  - 加入shortcut connections，形成网络的残差版本，每个残差模块有2个卷积层(图中实线)

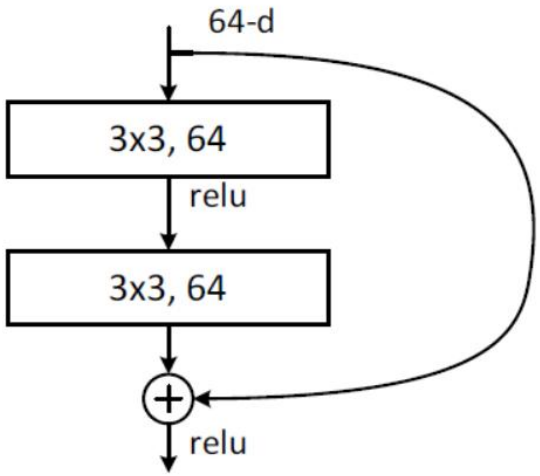


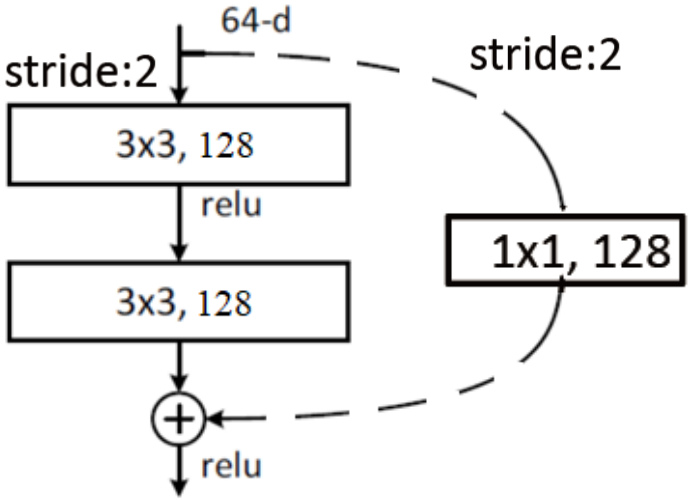


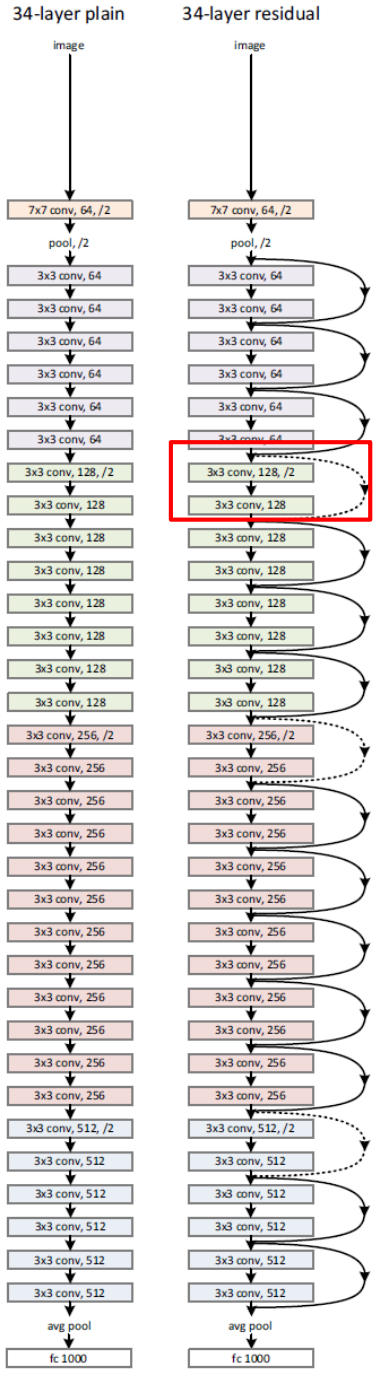
4）ResNet

- 网络结构(ResNet18/34)
  - 输入输出尺寸不同的处理：补零或使用1*1的卷积（图中虚线）



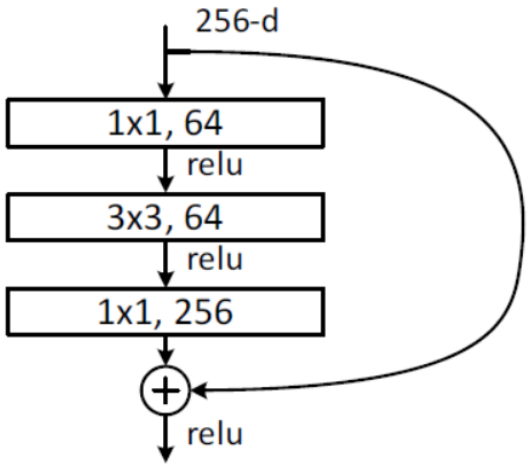


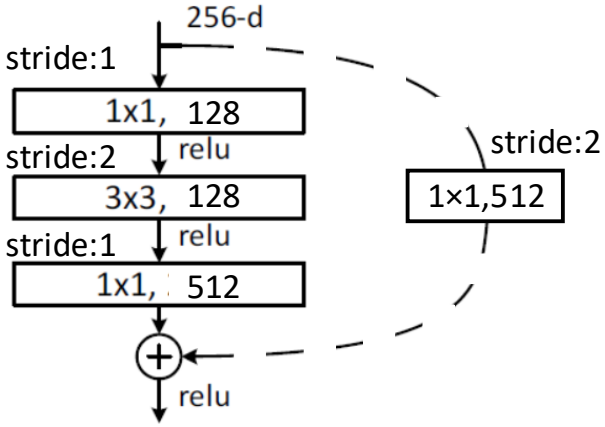




4）ResNet

- 网络结构(ResNet50/101/152)





layer name	output size	152-layer
conv1	112×112	
conv2_x	56×56	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix} \times 3$
	1×1	
FLOPs		11.3×10^9

4）ResNet

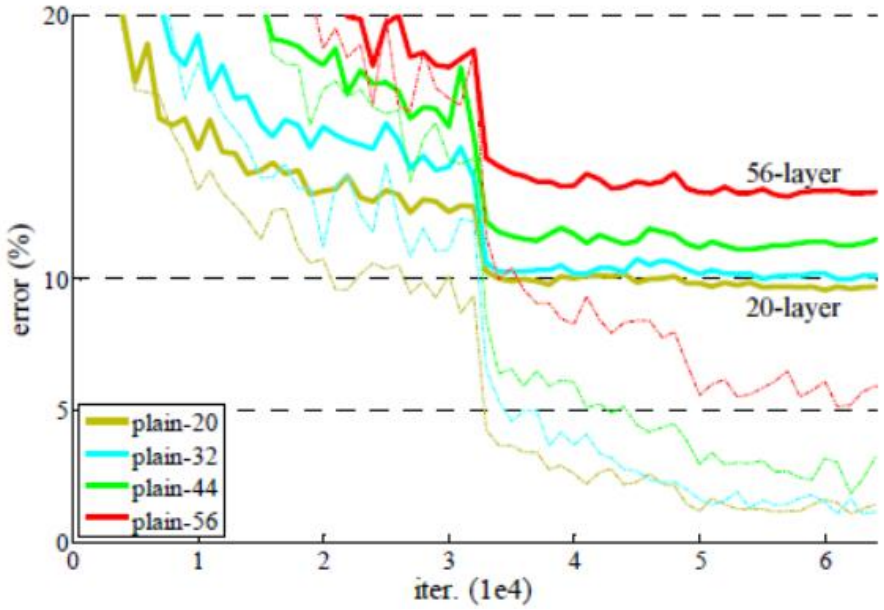
- 网络结构（5种）

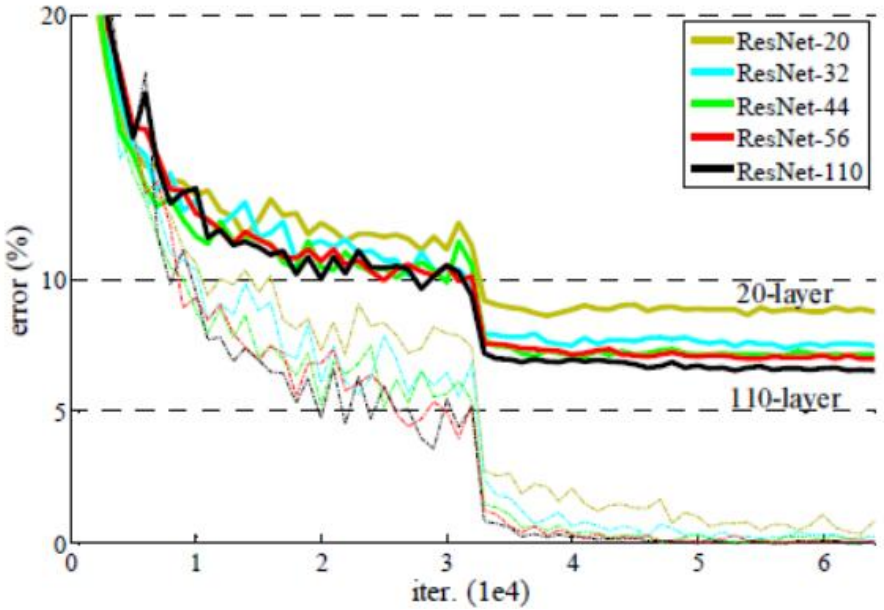
layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

4）ResNet

- **残差网络效果**
  - 训练深层网络不会出现退化
  - 深层网络训练误差更小

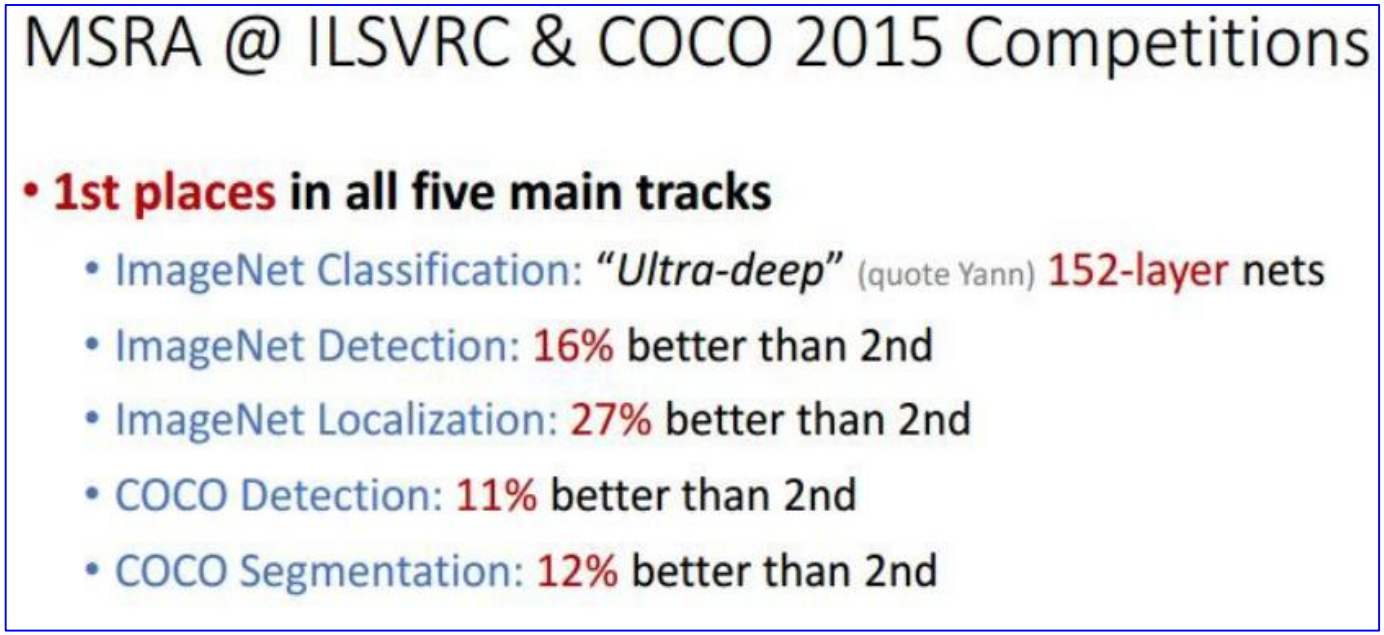
	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	**25.03**





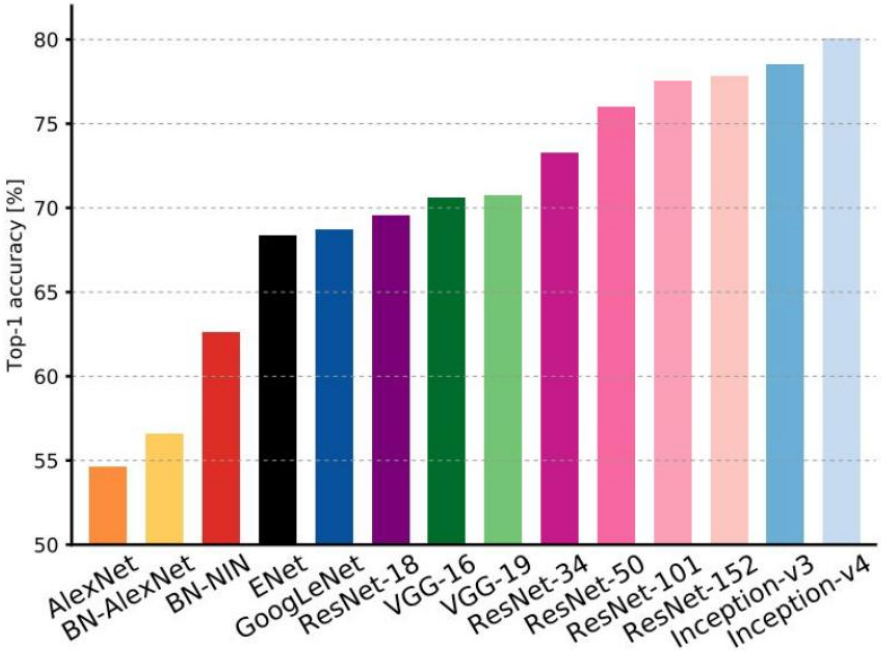
4）ResNet

- **残差网络效果**
  - 训练深层网络不会出现退化
  - 深层网络训练误差更小
  - 囊括了ILSVRC'15和COCO'15的所有分类和检测比赛冠军!

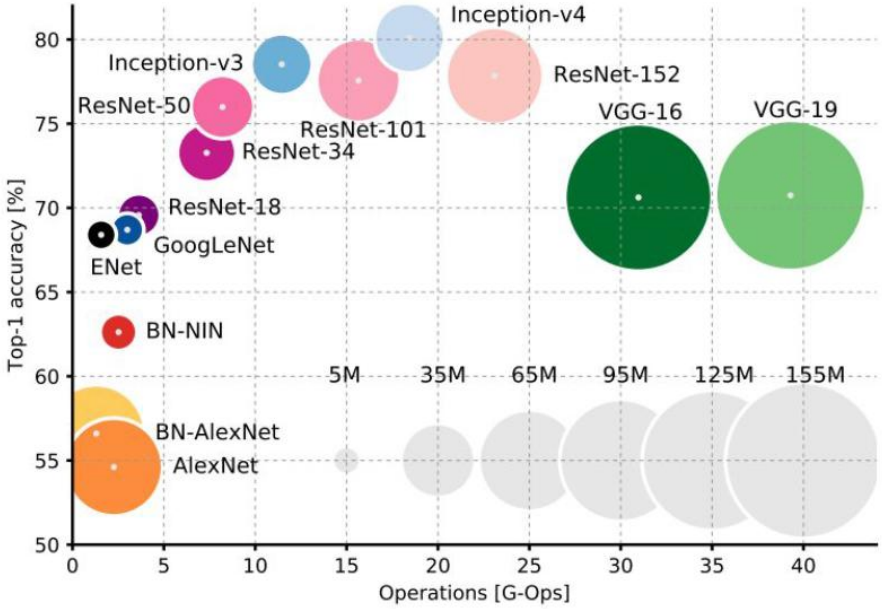


总结

- AlexNet ➡ 使用ReLu激活函数、引入dropout处理过拟合
- VGGNet ➡ 小卷积核（3*3）的使用
- GoogLeNet ➡ 引入Inception结构
- ResNet ➡ 引入残差学习



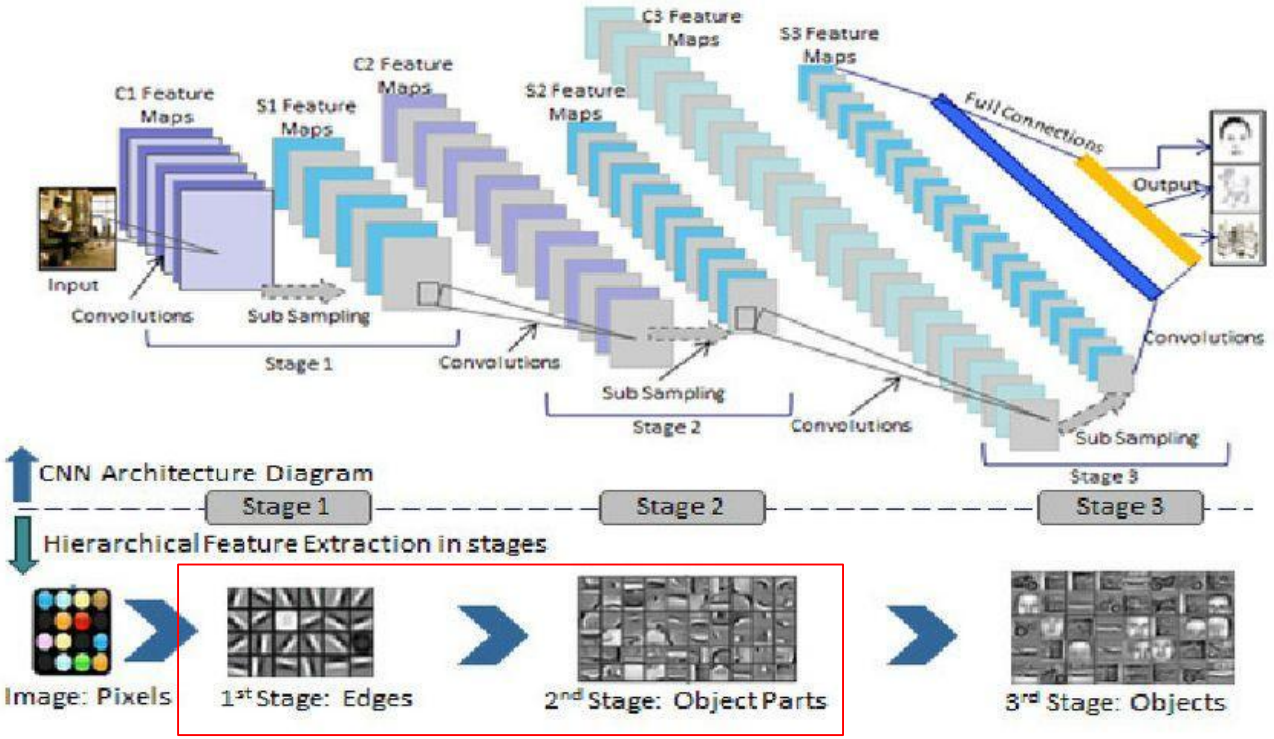
a）网络结构可达到的最高精度



b）网络结构精度vs计算量vs内存消耗

迁移学习(Transfer learning)

- 利用迁移学习，把已训练好的模型参数迁移到新的模型来帮助新模型训练，从而降低训练成本

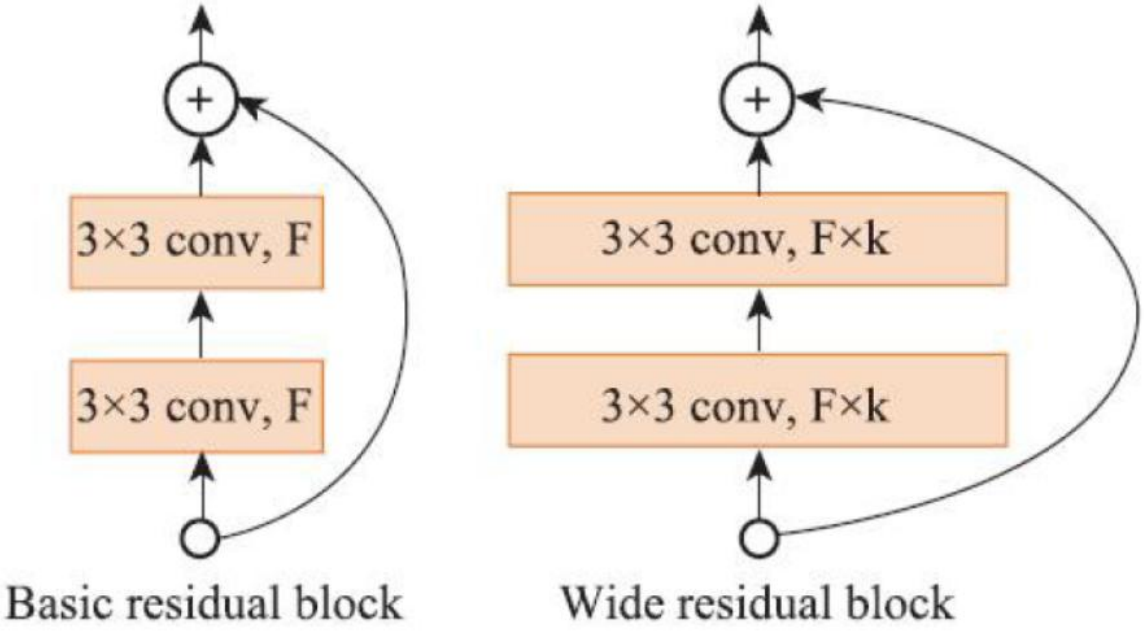


迁移学习(Transfer learning)

- 迁移学习方式

7.3 分类网络发展

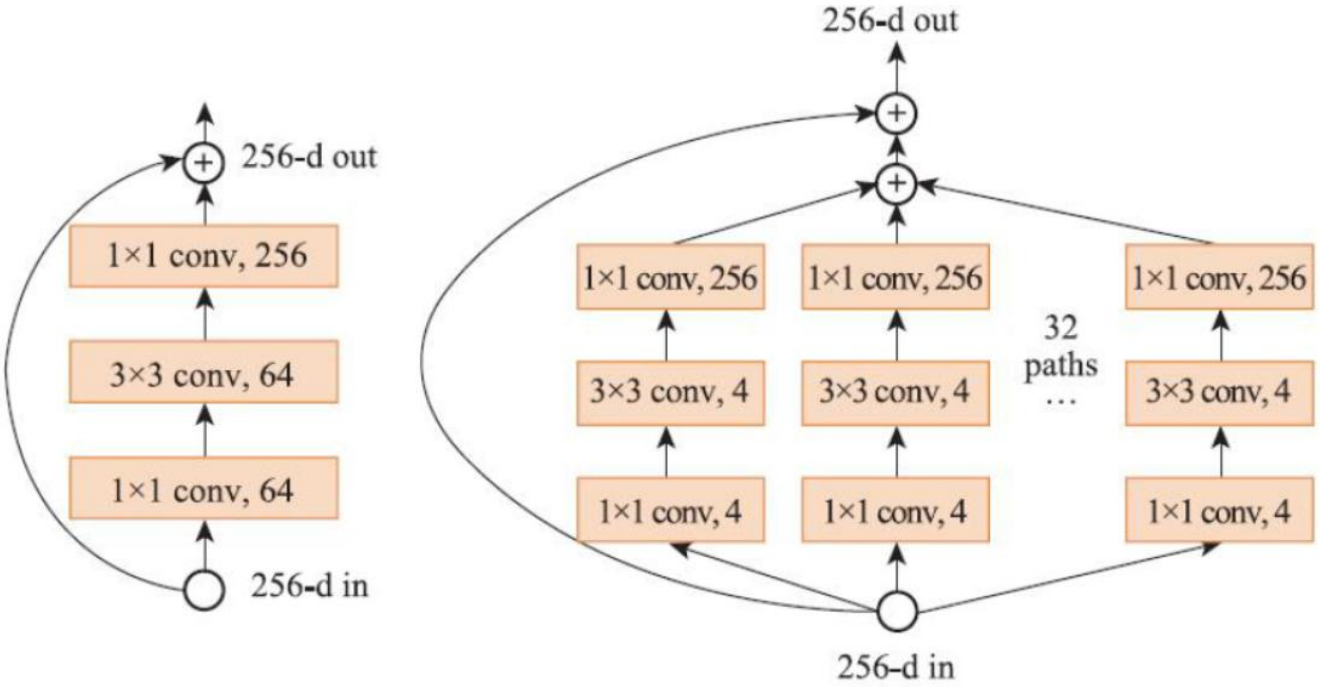
- **Wide ResNet**
  - Zagoruyko等人于2016年提出，其认为残差结构比深度更重要
  - 设计了更宽的残差模块，实验证明50层的加宽残差网络效果比152层的原ResNet网络效果更好



Basic residual block　　Wide residual block

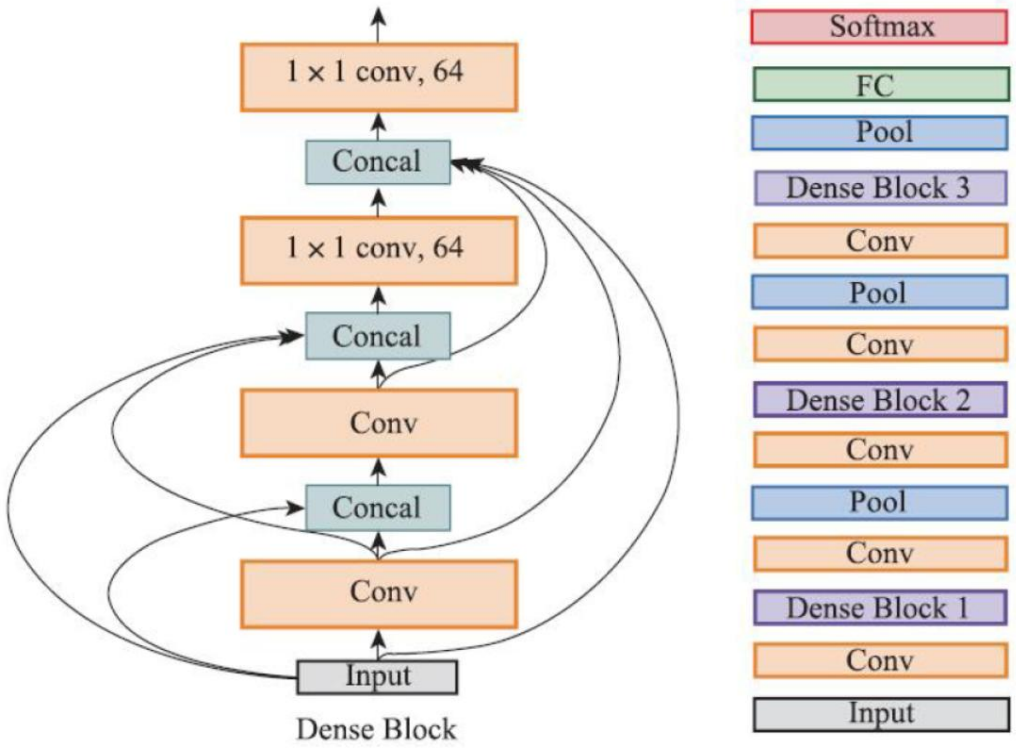
7.3 分类网络发展

- ResNeXT
  - 由Xie等人于2016年提出
  - 与Wide ResNet不同，ResNeXT在ResNet的基础上通过加宽inception个数的方式来扩展残差模块



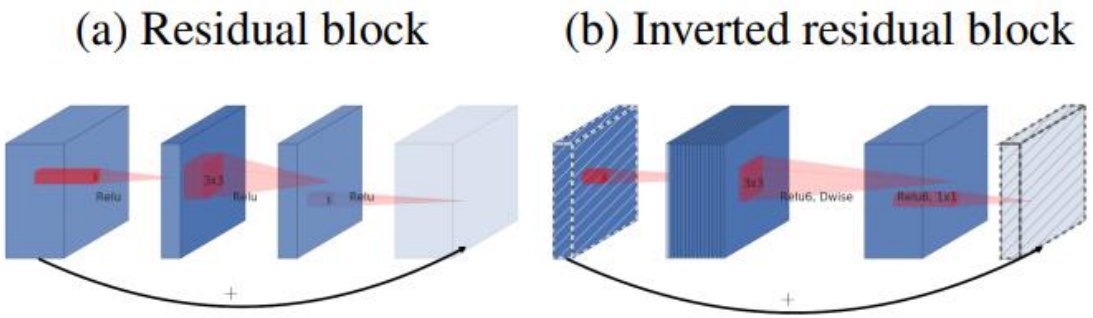
7.3 分类网络发展

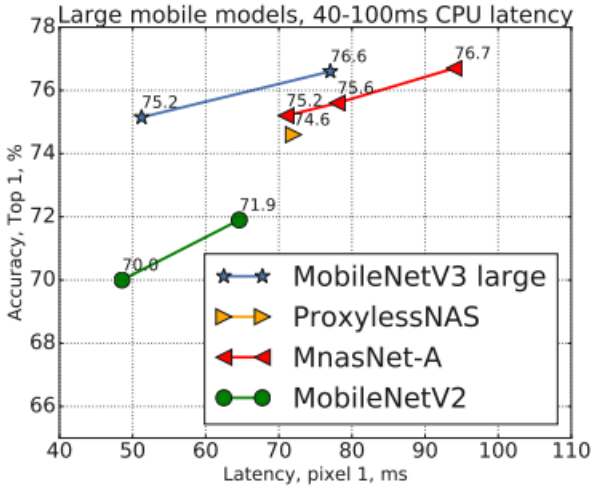
- DenseNet
  - 由Huang等人于2017年提出，每一层都与其他层相关联，这样的设计也大大缓解了“梯度消失”的问题



轻量级分类网络简介

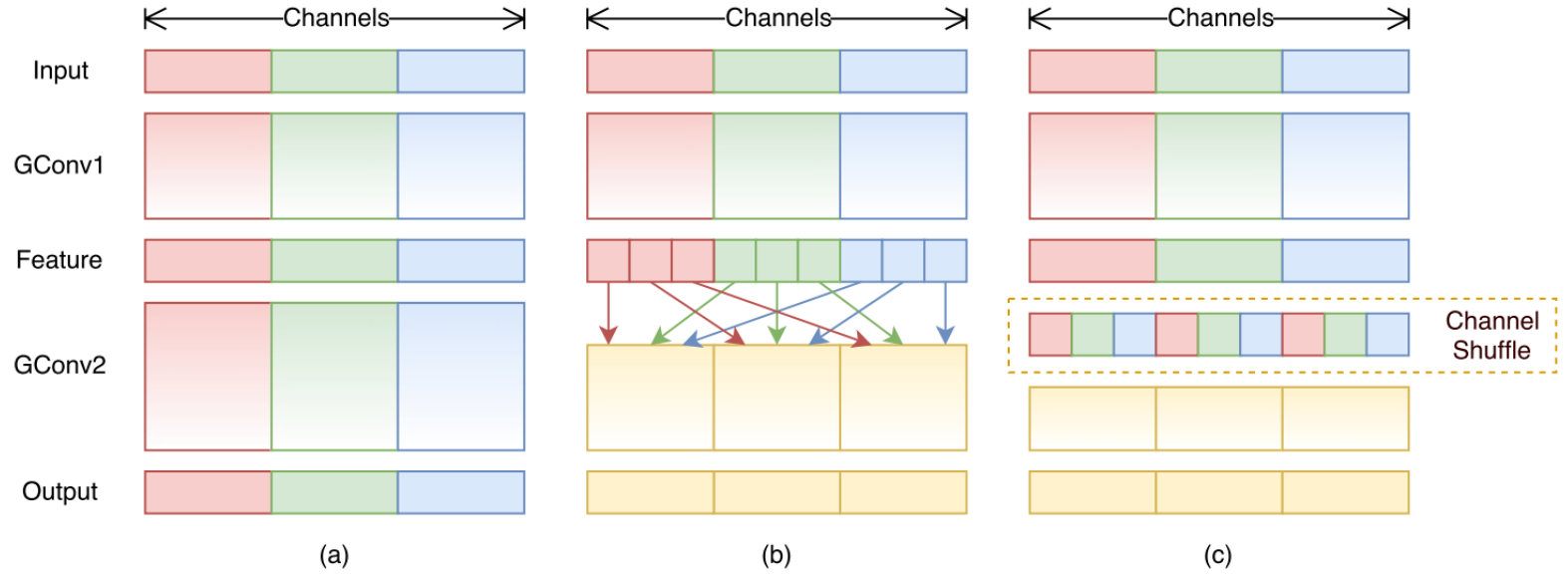
- 轻量级网络在尽量保持精度的前提下，从模型体积和推理速度两方面对网络进行轻量化改造
- **MobileNet**
  - v1(2017)：使用深度可分离卷积(dw/pw)构建网络
  - v2(2018)：使用倒残差结构和Linear Bottlenecks
  - v3(2019)：引入注意力机制，基于AutoML构建，再人工微调对搜索结果进行优化





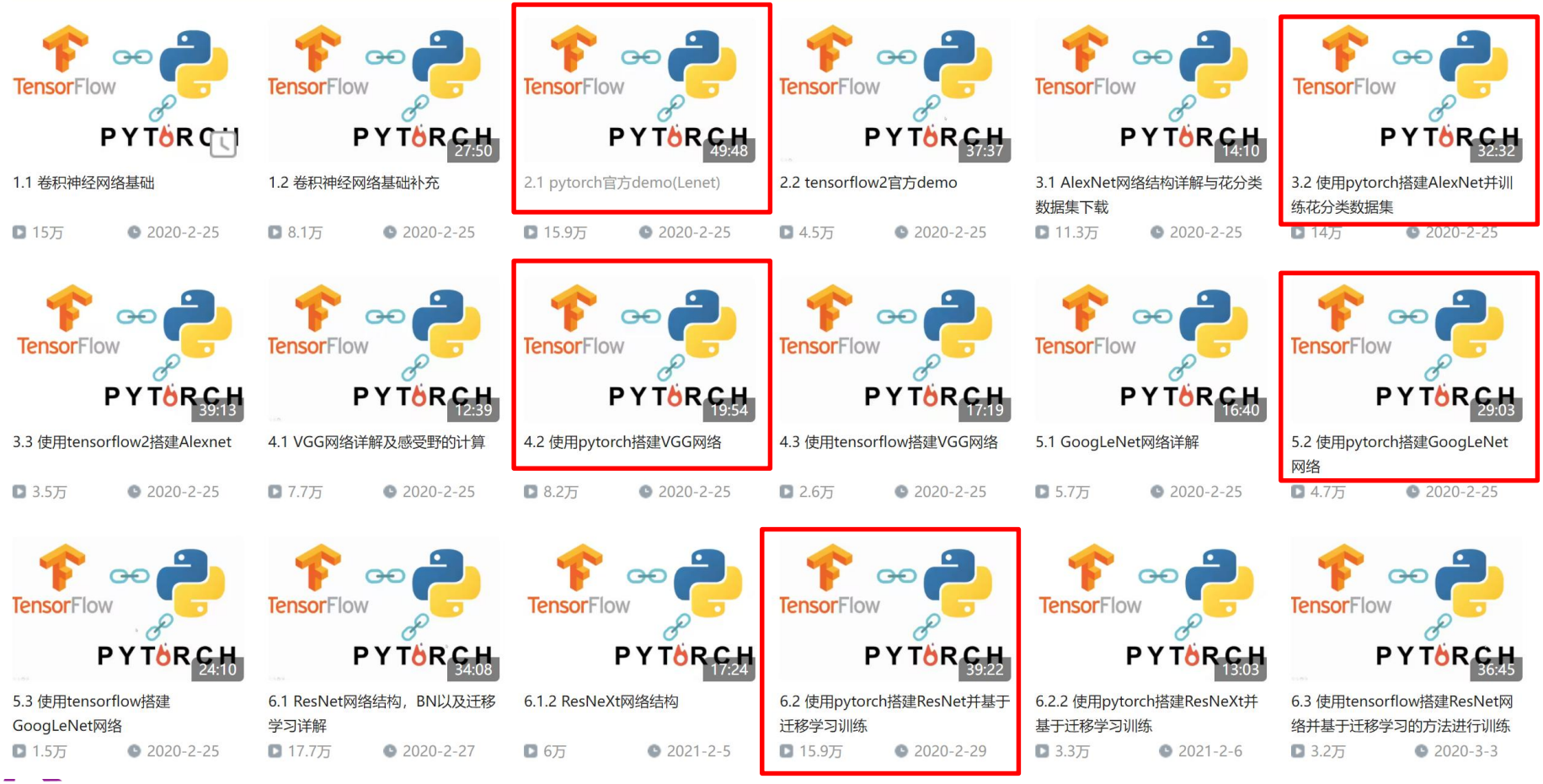
轻量级分类网络简介

- 轻量级网络在尽量保持精度的前提下，从模型体积和推理速度两方面对网络进行轻量化改造
- MobileNet
- ShuffleNet：提出了channel shuffle的思想
- SqueezeNet/Efficientnet……



课程编程讲解说明

- https://space.bilibili.com/18161609/channel/collectiondetail?sid=48290



The End

机器人与信息自动化研究所

Institute of Robotics & Automatic Information System

南開大學

Nankai University